

UNC
Fac. Ciencias Exactas
Físicas y Naturales

AED – Unidad 4

Análisis de Algoritmos

¿Cómo comparar dos algoritmos que resuelven un mismo problema en términos de eficiencia?

Podríamos implementar ambos algoritmos como programas informáticos y luego ejecutarlos con un rango adecuado de entradas, midiendo la cantidad de **recursos** en cuestión que utiliza cada programa. Este enfoque suele ser NO satisfactorio por cuatro razones:

Primero, está el esfuerzo que implica programar y probar dos algoritmos cuando, en el mejor de los casos, solo se desea conservar uno.

Segundo, al comparar empíricamente dos algoritmos, siempre existe la posibilidad de que uno de los programas esté "mejor escrito" que el otro y, por lo tanto, las cualidades relativas de los algoritmos subyacentes no estén realmente representadas por sus implementaciones.

Tercero, la elección de casos de prueba empíricos podría favorecer injustamente a un algoritmo.

Cuarto, podría descubrirse que incluso el mejor de los dos algoritmos no se ajusta a su presupuesto de **recursos**. En ese caso, debe comenzar todo el proceso de nuevo con otro programa que implemente un nuevo algoritmo

Análisis de Algoritmos

El **recurso** crítico para un programa suele ser su tiempo de ejecución. Sin embargo, no se puede prestar atención solo al tiempo de ejecución. También se deben considerar otros factores, como el espacio necesario para ejecutar el programa (tanto la memoria principal como el espacio en disco). Normalmente, se analiza el tiempo necesario para un algoritmo (o la instanciación de un algoritmo en forma de programa) y el espacio necesario para una estructura de datos.

Muchos factores afectan el tiempo de ejecución de un programa. Algunos factores se relacionan con el entorno en el que se compila y ejecuta el programa. Estos factores incluyen la velocidad de la CPU, el bus y el hardware periférico del ordenador. La competencia con otros usuarios por los recursos del ordenador (o de la red) puede ralentizar enormemente un programa.

El lenguaje de programación y la calidad del código generado por un compilador en particular pueden tener un efecto significativo. La eficiencia de codificación del programador que convierte el algoritmo en un programa también puede tener un impacto importante

Análisis de Algoritmos

Si realmente se desea comprender el tiempo de ejecución de un algoritmo, existen otros factores más apropiados a considerar que la velocidad de la máquina, el lenguaje de programación, el compilador, etc

La alternativa es utilizar **otra medida** como sustituto del tiempo de ejecución.

Al estimar el rendimiento de un algoritmo, una consideración fundamental es el número de operaciones básicas que requiere para procesar una entrada (datos) de cierto tamaño

Por ejemplo, al comparar algoritmos de ordenación, el tamaño del problema se mide típicamente por el número de registros a ordenar. Una operación básica debe tener la propiedad de que su tiempo de finalización no dependa de los valores particulares de sus operandos. Sumar o comparar dos variables enteras son ejemplos de operaciones básicas en la mayoría de los lenguajes de programación. Sumar el contenido de un array con n enteros no lo es, ya que el costo depende del valor de n (es decir, el tamaño de la entrada).

Estos problemas a menudo pueden evitarse mediante el **análisis asintótico**. El análisis asintótico mide la eficiencia de un algoritmo, o su implementación como programa, a medida que aumenta el tamaño de entrada (datos)

Análisis Asintótico

El análisis asintótico es el estudio del comportamiento de la complejidad de un algoritmo cuando el tamaño de la entrada n tiende a infinito. En lugar de medir tiempos exactos de ejecución (que dependen del hardware, compilador, lenguaje, etc.), se busca una medida independiente de factores externos, que muestre cómo escala el algoritmo a medida que aumenta el tamaño de los datos

Las características clave de esta definición son:

Comportamiento en el Límite: El enfoque principal no es el rendimiento para entradas pequeñas, sino cómo el algoritmo escala con entradas de gran tamaño (muchos datos del mismo tipo).

Se analiza la tasa de crecimiento de la función de complejidad.

Abstracción de Detalles: En la práctica, este análisis permite ignorar las constantes multiplicativas y los términos de menor orden en la función de complejidad. Esto se debe a que, para un n lo suficientemente grande, el término de mayor grado domina por completo el comportamiento de la función, haciendo que los otros términos sean insignificantes.

Enfoque en el Crecimiento: La meta es clasificar los algoritmos por su "orden de crecimiento", lo que permite predecir su rendimiento y determinar si son escalables para problemas complejos y grandes volúmenes de datos. En resumen, el análisis asintótico es el marco matemático para predecir la eficiencia de un algoritmo de forma independiente al entorno de ejecución, ofreciendo una visión clara de su escalabilidad y su idoneidad para resolver problemas a gran escala

Análisis Asintótico

La Esencia es la Simplificación: Descartando Términos y Constantes

Una de las prácticas más importantes en el análisis asintótico es la simplificación de la función de complejidad. Esto implica ignorar las constantes multiplicativas y los términos de orden inferior.

Por ejemplo, una función de tiempo de ejecución como $T(n)=3n^2+50n+100$ se simplificaría a $3n^2$. La razón es que, para un n lo suficientemente grande, el término de mayor grado en un polinomio domina por completo el comportamiento de la función.

Otro ejemplo, en la función $f(n)=4n^2+2n+7$, a medida que n crece, el término $4n^2$ contribuye de manera desproporcionadamente mayor a la suma total, haciendo que las contribuciones de $2n$ y 7 sean insignificantes.

Las notaciones asintóticas, con sus definiciones formales, validan esta simplificación al establecer que el crecimiento de una función puede ser acotado por un múltiplo constante de un término de crecimiento dominante a partir de **un cierto punto**.

Análisis Asintótico

Las funciones asintóticas describen el comportamiento de un algoritmo cuando el tamaño de entrada tiende a infinito. Nos permiten clasificar algoritmos según su eficiencia sin depender de factores externos como hardware o lenguaje de programación.

¿Por qué son importantes?

- Predicen rendimiento a gran escala
- Comparan algoritmos objetivamente
- Ignoran detalles irrelevantes (constantes, hardware)
- Se enfocan en el crecimiento de la función

Las Tres Notaciones Fundamentales

Notación	Nombre	Significado	Describe
O	Big O (O grande)	Límite superior	Peor caso / Cota superior
Ω	Big Omega	Límite inferior	Mejor caso / Cota inferior
Θ	Big Theta	Límite ajustado	Caso promedio / Cota ajustada

Análisis Asintótico

Notación Big O (O Grande) - Límite Superior

Definición Formal

$f(n) = O(g(n))$ si y solo si existen constantes positivas c y n_0 tales que:

$$f(n) \leq c \cdot g(n) \text{ para todo } n \geq n_0$$

Interpretación

- $f(n)$ crece a lo sumo como $g(n)$

- Describe el peor caso del algoritmo

- Es un límite superior: el algoritmo nunca será peor que esto

Análisis Asintótico - Big O

Ejemplo Búsqueda Lineal:

```
int busqueda_lineal(int arr[], int n, int x) {  
    for (int i = 0; i < n; i++) {        // Máximo n iteraciones  
        if (arr[i] == x) return i;      // O(1) por iteración  
    }  
    return -1;  
}
```

Análisis: En el peor caso (elemento no está), recorre todo el arreglo.

Complejidad: **$O(n)$**

La búsqueda lineal tiene una complejidad $O(n)$

Análisis Asintótico

Notación Big Omega (Ω) - Límite Inferior

Definición Formal

$f(n) = \Omega(g(n))$ si y solo si existen constantes positivas c y n_0 tales que:

$$f(n) \geq c \cdot g(n) \text{ para todo } n \geq n_0$$

Interpretación

- "f(n) crece al menos como g(n)"
- Describe el mejor caso del algoritmo
- Es un límite inferior: el algoritmo nunca será mejor que esto

Análisis Asintótico - Big Ω

Ejemplo Búsqueda Lineal:

```
int busqueda_lineal(int arr[], int n, int x) {  
    for (int i = 0; i < n; i++) {  
        if (arr[i] == x) return i;    // Mejor caso: primera posición  
    }  
    return -1;  
}
```

Análisis: En el mejor caso, encuentra el elemento en la primera posición.
Complejidad: **$\Omega(1)$**

La búsqueda lineal tiene una complejidad $\Omega(1)$

La búsqueda lineal tiene una complejidad $O(n)$

Análisis Asintótico

Notación Big Theta (Θ) - Límite Ajustado

Definición Formal

$f(n) = \Theta(g(n))$ si y solo si:

$$f(n) = O(g(n)) \text{ and } f(n) = \Omega(g(n))$$

Es decir, existen constantes c_1 , c_2 y n_0 tales que:

$$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \text{ para todo } n \geq n_0$$

Interpretación

- "f(n) crece exactamente como g(n)"
- El mejor y peor caso tienen el mismo orden de crecimiento
- Es el límite más preciso

Análisis Asintótico - Big Θ

Ejemplo Algoritmo de Ordenamiento Burbuja:

```
void burbuja(int arr[], int n) {  
    for (int i = 0; i < n-1; i++) {          // n-1 iteraciones  
        for (int j = 0; j < n-i-1; j++) {    // Varía de n-1 a 1  
            if (arr[j] > arr[j+1]) {  
                swap(arr[j], arr[j+1]);  
            }  
        }  
    }  
}
```

Análisis: **Siempre** realiza aproximadamente $n^2/2$ comparaciones, sin importar el orden inicial. Complejidad: **$\Theta(n^2)$**

El método de la burbuja es $\Omega(n^2)$

El método de la burbuja es $O(n^2)$

por lo tanto es $\Theta(n^2)$

Análisis Asintótico

Ejemplo: Multiplicación de Matrices

```
void multiplicar_matrices(int A[N][N], int B[N][N], int C[N][N]) {  
    for (int i = 0; i < N; i++) {      // N iteraciones  
        for (int j = 0; j < N; j++) {  // N iteraciones  
            C[i][j] = 0;  
            for (int k = 0; k < N; k++) { // N iteraciones  
                C[i][j] += A[i][k] * B[k][j];  
            }  
        }  
    }  
}
```

Análisis: **Siempre** realiza exactamente n^3 multiplicaciones.

Notación

$O(n^3)$ Límite superior

$\Omega(n^3)$ Límite inferior

$\Theta(n^3)$ Límite ajustado <<<<

UNC
Fac. Ciencias Exactas
Físicas y Naturales

AED – Ordenamiento

Merge Sort

Un enfoque natural para la resolución de problemas es “divide y vencerás”. En cuanto al ordenamiento, podríamos considerar dividir la lista (a ordenar) en partes, procesarlas (ordenarlas) y luego volver a unir las. La operación fundamental de este algoritmo es fusionar dos listas ordenadas

El algoritmo Mergesort se ejecuta en un tiempo de ejecución de $O(N \log N)$, en el peor de los casos, y el número de comparaciones utilizadas es casi óptimo. Es un buen ejemplo de algoritmo recursivo.

Algorithm for Merge Sort

We can modify merging to serve as an approach to sorting a single, unsorted table as follows:

1. Split the table into two halves.
2. Sort the left half.
3. Sort the right half.
4. Merge the two.

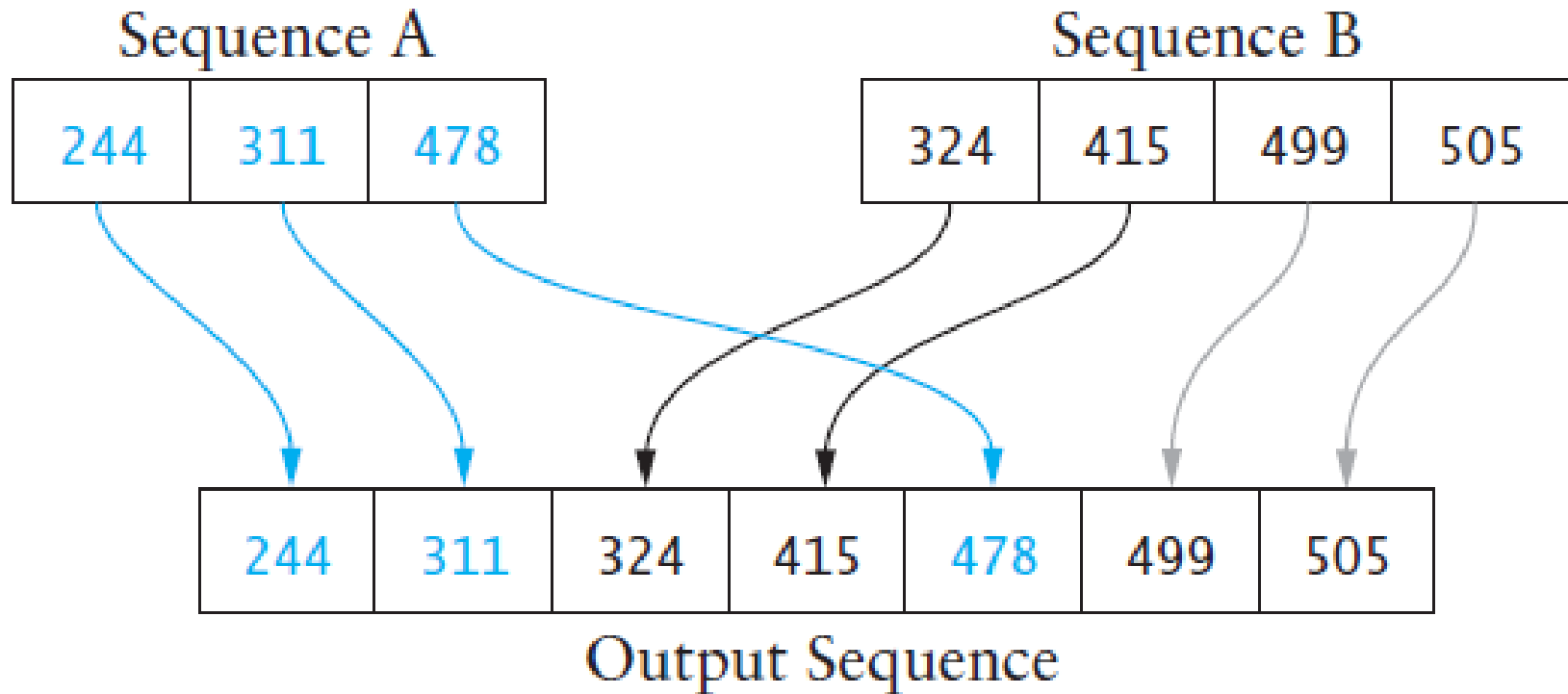
Algorithm for Merge Sort

1. **if** the table has more than one element
2. Store the first half of the table in `left_table`.
3. Store the second half of the table in `right_table`.
4. Recursively apply the merge sort algorithm to `left_table`.
5. Recursively apply the merge sort algorithm to `right_table`.
6. Call the merge function with `left_table` and `right_table` as the input sequences and the original table as the output sequence.

Merge Algorithm

1. Access the first item from both sequences.
2. **while** not finished with either sequence
3. Compare the current items from the two sequences, copy the smaller current item to the output sequence, and access the next item from the input sequence whose item was copied.
4. Copy any remaining items from the first sequence to the output sequence.
5. Copy any remaining items from the second sequence to the output sequence.

Merge Sort <https://www.youtube.com/watch?v=A215cCriLy4>



Heap Sort

El **Heapsort** es un algoritmo de ordenamiento basado en comparaciones que utiliza la estructura de datos **heap binario** para ordenar una colección de elementos. Se considera un algoritmo eficiente con una complejidad temporal garantizada. Consta de dos fases principales:

1. Construir el Heap (Heapify):

1. La primera fase consiste en transformar la lista o array de entrada en un **heap binario máximo**, el cual es una estructura de árbol binario completa donde el valor de cada nodo padre es mayor o igual que el valor de sus nodos hijos. El **elemento más grande siempre está en la raíz** del árbol.
2. Este proceso generalmente se realiza "in-place", dentro del mismo array, considerando al array como la representación de un árbol binario.
3. Construir el heap máximo a partir de un array desordenado toma **$O(n)$** tiempo.

Original Numbers

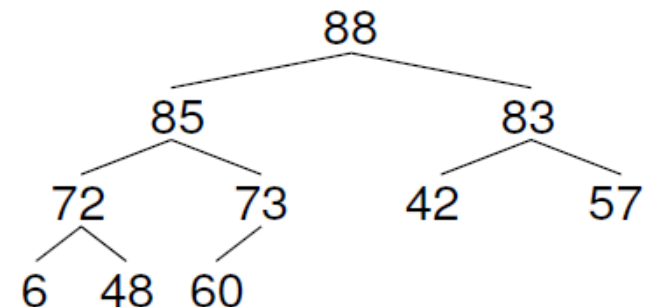
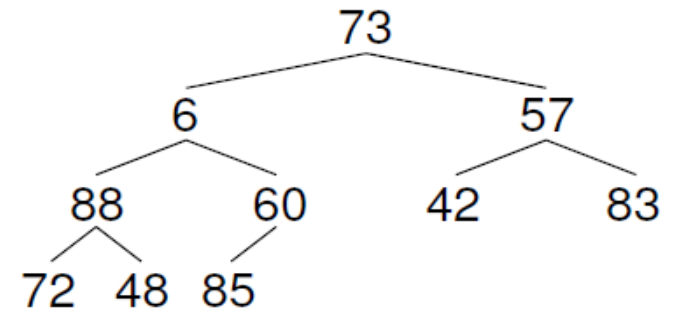
73	6	57	88	60	42	83	72	48	85
----	---	----	----	----	----	----	----	----	----

$A[i] \geq A[2*i+1]$ hijo izq

$A[i] \geq A[2*i+2]$ hijo der

Build Heap

88	85	83	72	73	42	57	6	48	60
----	----	----	----	----	----	----	---	----	----



2. Ordenar Extrayendo Elementos:

1. Una vez que se ha construido el heap máximo, se intercambia el elemento raíz (el más grande) con el **último elemento** del heap (que está en la última posición de la parte no ordenada del array).
2. Después del intercambio, el último elemento del array está en su posición final ordenada. La parte del array que aún es parte del heap se reduce en uno.
3. Ahora, el nuevo elemento en la raíz puede no cumplir la propiedad de heap máximo, por lo que se realiza una operación de **"heapify"** en la nueva raíz para restaurar la propiedad de heap en el subárbol restante. Esta operación toma **$O(\log n)$** tiempo, donde n es el tamaño actual del heap.
4. Estos pasos se repiten $n-1$ veces (una vez por cada elemento, excepto el último) hasta que el heap está vacío.

Remove 88

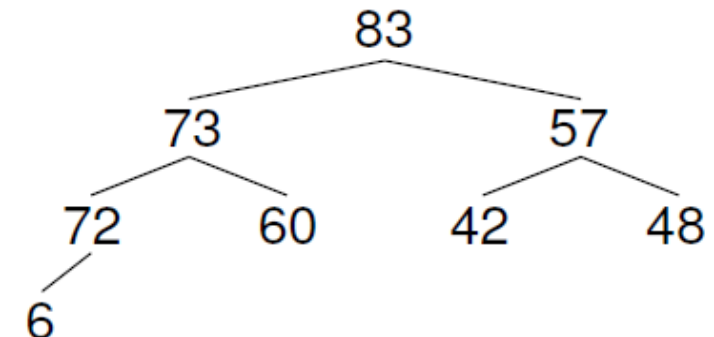
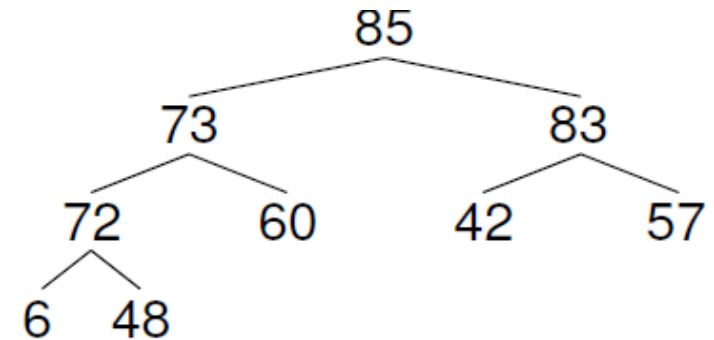
85	73	83	72	60	42	57	6	48	88
----	----	----	----	----	----	----	---	----	----

$A[i] \geq A[2*i+1]$ hijo izq

$A[i] \geq A[2*i+2]$ hijo der

Remove 85

83	73	57	72	60	42	48	6	85	88
----	----	----	----	----	----	----	---	----	----



Algorithm for In-Place Heapsort

1. Build a heap by rearranging the elements in an unsorted array.
2. **while** the heap is not empty
3. Remove the first item from the heap by swapping it with the last item in the heap and restoring the heap property.

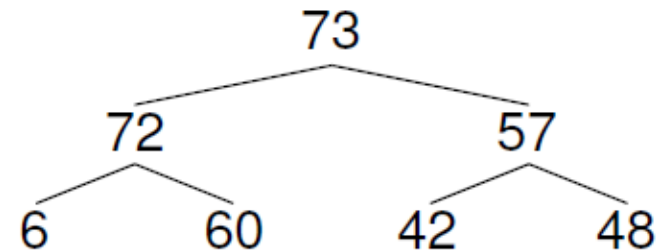
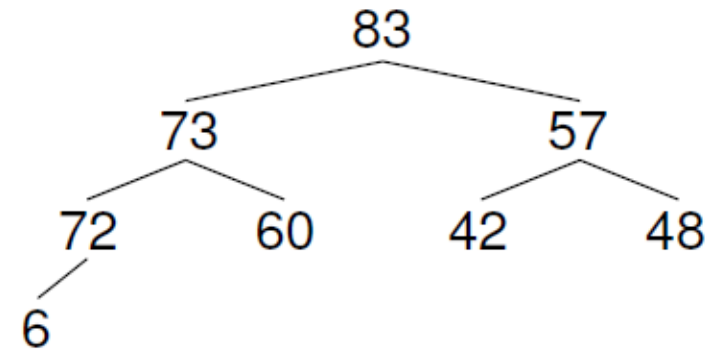
Remove 85

83	73	57	72	60	42	48	6	85	88
----	----	----	----	----	----	----	---	----	----

$A[i] \geq A[2*i+1]$ <- hijo izq padre en $(i-1) / 2$
 $A[i] \geq A[2*i+2]$ <- hijo der

Remove 83

73	72	57	6	60	42	48	83	85	88
----	----	----	---	----	----	----	----	----	----

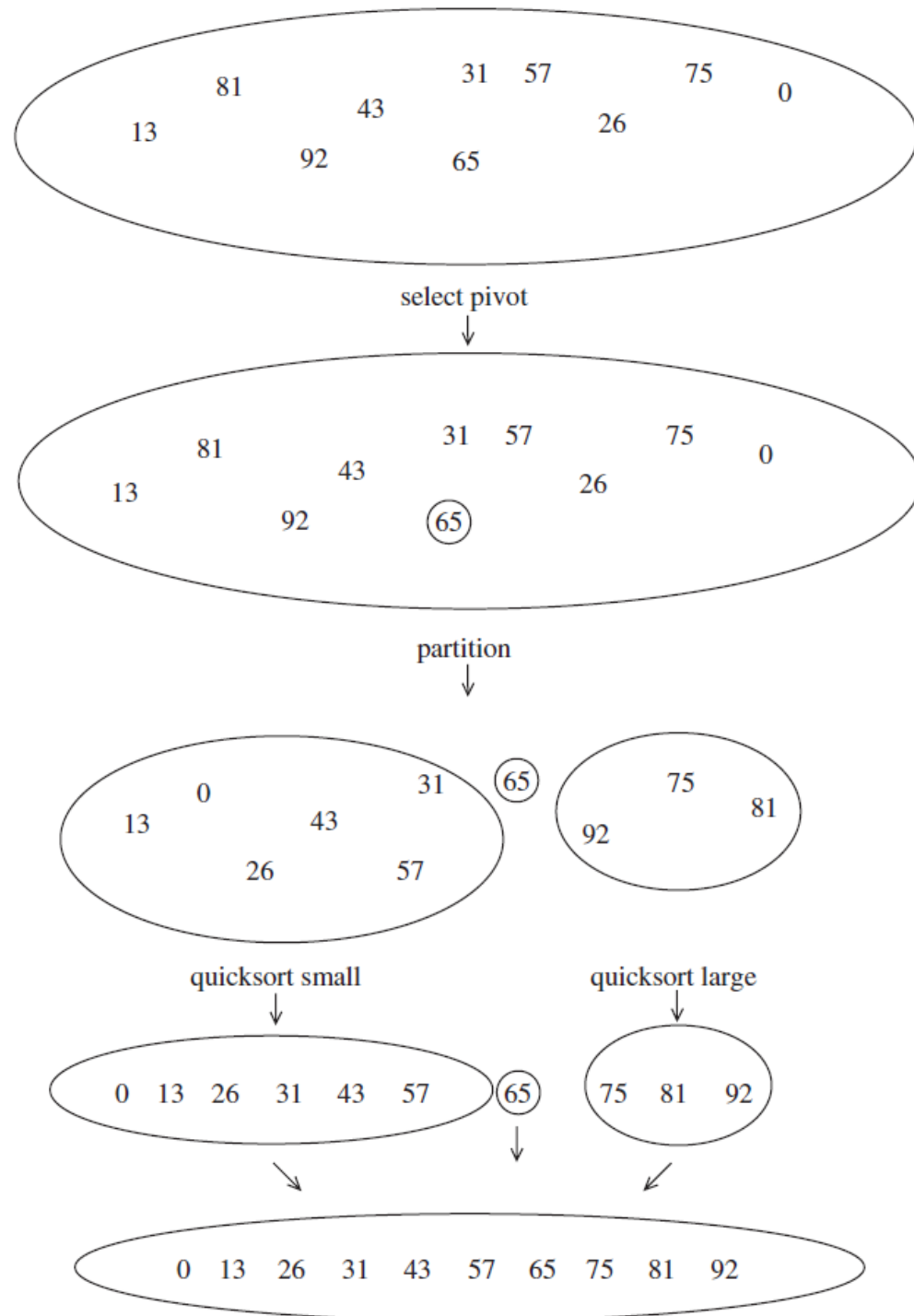


Quick Sort (divide y vencerás)

Dado un arreglo A

- 1- Si el número de elementos de A es 0 o 1, terminar
- 2- Elegir un elemento cualquiera de A al que llamaremos **pivote**
- 3- Particionar A – {pivote} en dos grupos disjuntos
$$A1 = \{ x \in A - \{\text{pivote}\} \mid x \leq \text{pivote} \}$$
$$A2 = \{ x \in A - \{\text{pivote}\} \mid x \geq \text{pivote} \}$$
- 4- Repetir este proceso de forma recursiva para cada subarreglo mientras éstos contengan más de un elemento. Una vez terminado este proceso todos los elementos estarán ordenados

Quick Sort (divide y vencerás)



Quicksort

30	45	130	70	100	150	49	140	18	110
i									j

30	45	130	70	100	150	49	140	18	110
		i						j	

30	45	18	70	100	150	49	140	130	110
		i						j	

Quicksort

30	45	130	70	100	150	49	140	18	110
----	----	-----	----	-----	-----	----	-----	----	-----

i

j

30	45	130	70	100	150	49	140	18	110
----	----	-----	----	-----	-----	----	-----	----	-----

i

j

30	45	18	70	100	150	49	140	130	110
----	----	----	----	-----	-----	----	-----	-----	-----

i

j

30	45	130	70	100	150	49	140	18	110
----	----	-----	----	-----	-----	----	-----	----	-----

i

j

30	45	130	70	100	49	150	140	18	110
----	----	-----	----	-----	----	-----	-----	----	-----

i

j

Quicksort

30	45	130	70	100	150	49	140	18	110
----	----	-----	----	-----	-----	----	-----	----	-----

i

j

30	45	130	70	100	150	49	140	18	110
----	----	-----	----	-----	-----	----	-----	----	-----

i

j

30	45	18	70	100	150	49	140	130	110
----	----	----	----	-----	-----	----	-----	-----	-----

i

j

30	45	18	70	100	150	49	140	130	110
----	----	----	----	-----	-----	----	-----	-----	-----

i

j

30	45	18	70	100	49	150	140	130	110
----	----	----	----	-----	----	-----	-----	-----	-----

i

j

30	45	18	70	100	49	150	140	130	110
----	----	----	----	-----	----	-----	-----	-----	-----

j

i

ultimo

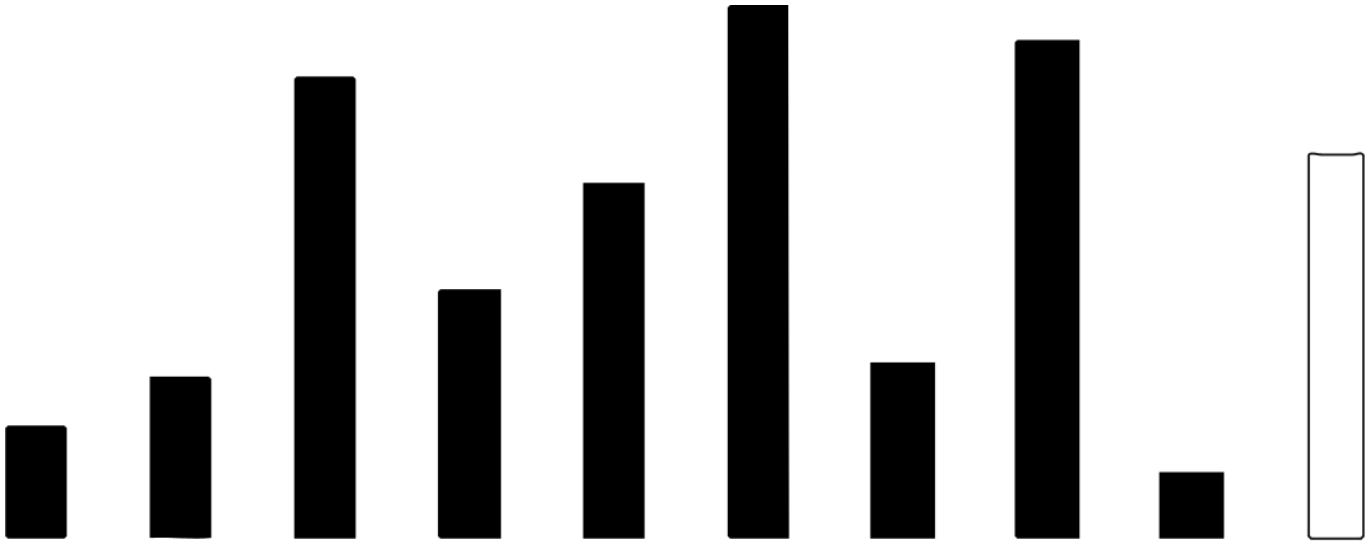
30	45	18	70	100	49	110	140	130	150
----	----	----	----	-----	----	-----	-----	-----	-----

j

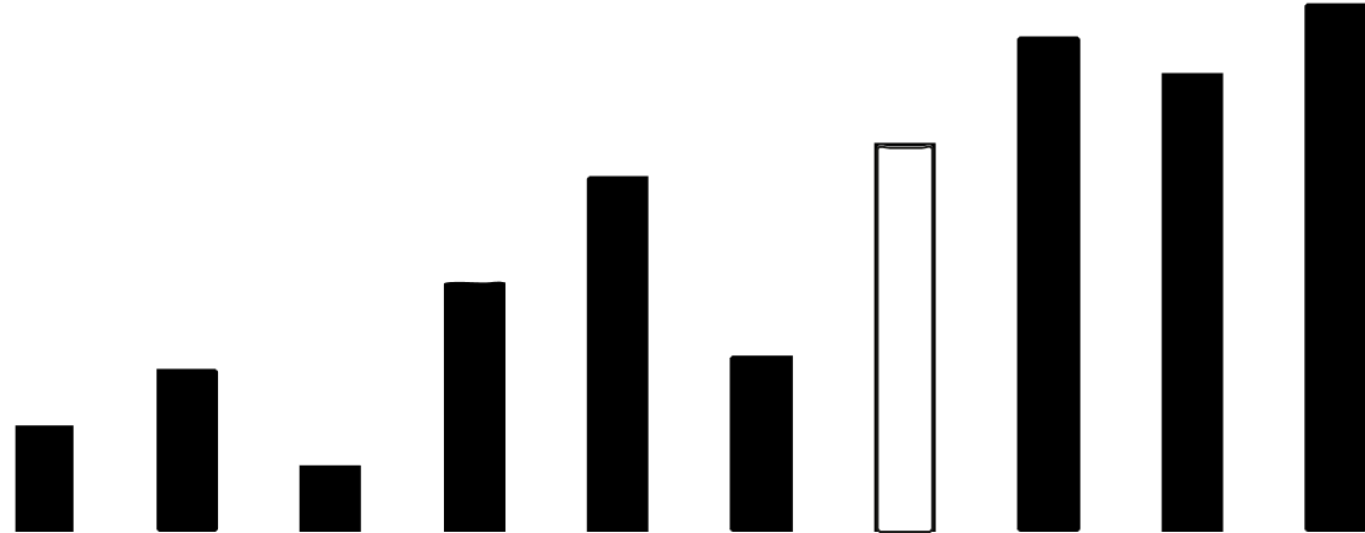
i

ultimo

30	45	130	70	100	150	49	140	18	110
----	----	-----	----	-----	-----	----	-----	----	-----



30	45	18	70	100	49	110	140	130	150
----	----	----	----	-----	----	-----	-----	-----	-----



Quicksort

Mejoras

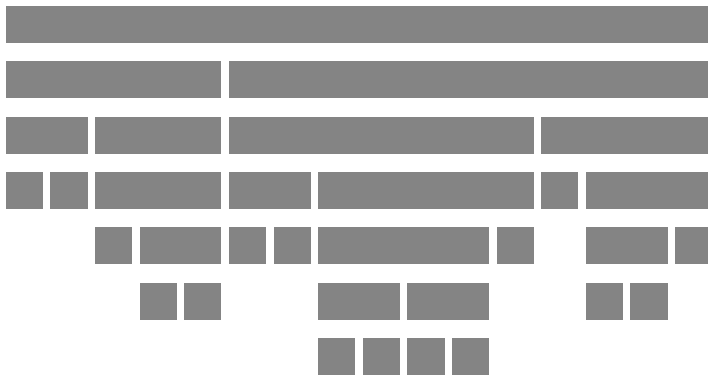
- 1- Aplicando un ordenamiento por inserción a los subarray menores de 20 elementos
- 2- Utilizando como pivote la mediana de tres elementos del subarray ej: $A[\text{primero}]$, $A[\text{ultimo}]$ y $A[(\text{primero}+\text{ultimo})/2]$
- 3- Implementandolo en forma no recursiva
- 4- Utilizar como pivote la media de cada subarray (**meansort**)
- 5- Utilizar como pivote el elemento intermedio del subarray y durante la partición cuando se buscan de izq a der los valores $>$ pivote se intercambian los elementos del array de modo que a la izq van quedando los elementos mas pequeños, y viceversa cuando se buscan de der a izq los valores $<$ pivote. (**Bsort**)

En promedio se logran mejoras entre un 20% a un 30%

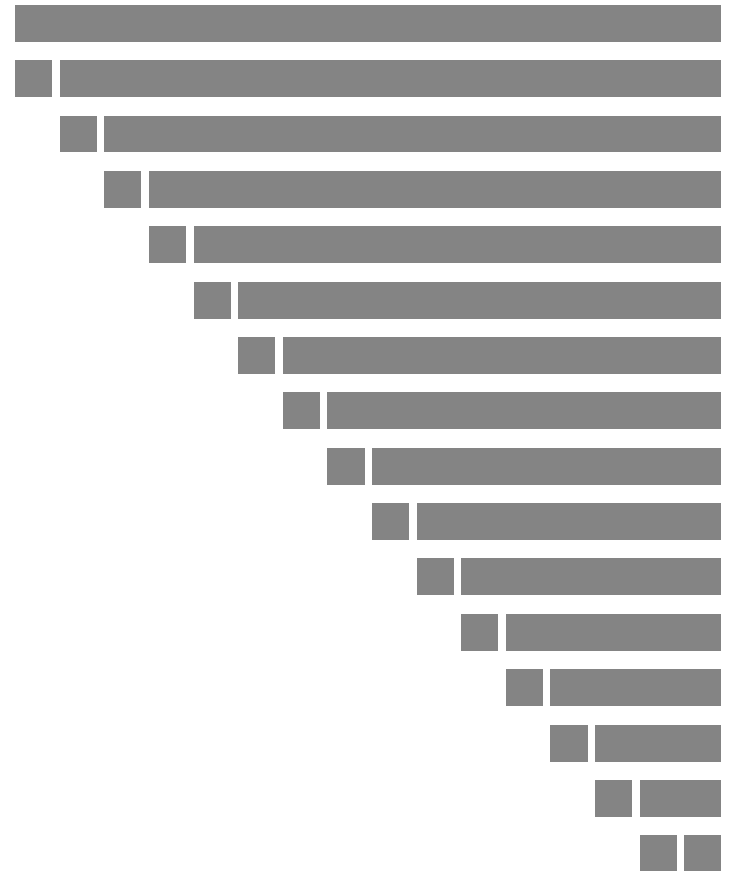
Quicksort



(a)



(b)



(c)

Recursion depth of Quicksort: a) best case, b) average case, c) worst case

QuickSort

https://www.youtube.com/watch?v=uf_DyA_Ku7A

Simulación de los distintos métodos de ordenamiento

<https://www.toptal.com/developers/sorting-algorithms>

Tabla Comparativa para Selección Según Parámetros

Tabla 1: Complejidades y Características Básicas

Algoritmo	Mejor Caso	Caso Promedio	Peor Caso	Espacio	Estable	In-Place
Burbuja	$O(n)^1$	$O(n^2)$	$O(n^2)$	$O(1)$	✓ Sí	✓ Sí
Selección	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	✗ No	✓ Sí
Inserción	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✓ Sí	✓ Sí
Shell	$O(n \log n)$	$O(n^{1.25})$	$O(n^2)$	$O(1)$	✗ No	✓ Sí
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	✓ Sí	✗ No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	✗ No	✓ Sí
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)^2$	✗ No	✓ Sí

Notas:

- ¹ Con optimización de detección temprana
- ² Stack de recursión promedio

Tabla Comparativa para Selección Según Parámetros

Tabla 2: Selección por Tamaño de Datos

Tamaño de Datos	Algoritmos Recomendados	Algoritmos a Evitar	Justificación
Muy pequeño ($n < 10$)	Inserción, Burbuja	Merge Sort, Heap Sort	El overhead de algoritmos complejos no compensa
Pequeño ($10 \leq n < 50$)	Inserción, Selección	Quick Sort, Merge Sort	Algoritmos simples son más eficientes
Mediano ($50 \leq n < 1000$)	Shell, Quick Sort	Burbuja, Selección	Balance entre simplicidad y eficiencia
Grande ($1000 \leq n < 100k$)	Quick Sort, Merge Sort	Burbuja, Selección, Inserción	Necesario $O(n \log n)$
Muy grande ($n \geq 100k$)	Merge Sort, Heap Sort	Todos los $O(n^2)$	Comportamiento garantizado $O(n \log n)$

Tabla Comparativa para Selección Según Parámetros

Tabla 3: Selección según Estado Inicial de los Datos

Estado de los Datos	Mejor Opción	Segunda Opción	Peor Opción	Observaciones
Ya ordenados	Inserción $O(n)$	Burbuja $O(n)$	Selección $O(n^2)$	Aprovecha el orden existente
Casi ordenados	Inserción	Shell	Quick Sort	Pocas inversiones a corregir
Ordenados inverso	Merge Sort	Heap Sort	Inserción $O(n^2)$	Comportamiento predecible
Aleatorios	Quick Sort	Merge Sort	Burbuja	Caso promedio favorable
Con duplicados	Merge Sort	Inserción	Quick Sort ³	Estabilidad importante
Datos únicos	Quick Sort	Heap Sort	Burbuja	Aprovecha la distribución

Nota: ³ Quick Sort puede degradarse con muchos duplicados

Tabla Comparativa para Selección Según Parámetros

Tabla 4: Selección por Restricciones de Memoria

Restricción de Memoria	Algoritmos Permitidos	Algoritmos Prohibidos	Motivo
Muy limitada	Selección, Heap Sort	Merge Sort	No pueden permitirse $O(n)$ extra
Limitada	Inserción, Shell, Quick Sort	Merge Sort	Prefieren $O(1)$ o $O(\log n)$
Abundante	Merge Sort	Ninguno	Puede aprovechar espacio extra
Sistema embebido	Inserción, Selección	Quick Sort ⁴	Stack de recursión impredecible

Nota: ⁴ A menos que se implemente iterativamente

Tabla Comparativa para Selección Según Parámetros

Tabla 5: Selección por Requerimientos Especiales

Requerimiento	Algoritmos que lo Cumplen	Algoritmos que NO	Casos de Uso
Estabilidad	Merge Sort, Inserción, Burbuja	Selección, Shell, Heap Sort, Quick Sort	Registros con múltiples claves
In-Place	Heap Sort, Quick Sort, Shell, Inserción, Selección, Burbuja	Merge Sort	Memoria limitada
Comportamiento garantizado	Merge Sort, Heap Sort	Quick Sort	Sistemas de tiempo real
Fácil implementación	Burbuja, Selección, Inserción	Merge Sort, Heap Sort, Shell	Prototipado rápido
Adaptativo	Inserción, Burbuja	Selección, Heap Sort, Merge Sort	Datos parcialmente ordenados

Tabla Comparativa para Selección Según Parámetros

Tabla 6: Guía de Decisión Rápida

Situación	Pregunta Clave	Algoritmo Recomendado
Datos pequeños	¿ $n < 50$?	→ Inserción
Necesito estabilidad	¿Mantengo orden relativo?	→ Merge Sort
Memoria muy limitada	¿Solo $O(1)$ espacio extra?	→ Heap Sort
Datos casi ordenados	¿Pocas inversiones?	→ Inserción
Rendimiento promedio	¿Datos aleatorios grandes?	→ Quick Sort
Peor caso garantizado	¿Tiempo crítico?	→ Merge Sort
Simplicidad	¿Fácil de implementar/debuggear?	→ Selección

Tabla Comparativa para Selección Según Parámetros

Tabla 7: Análisis por Tipo de Aplicación

Tipo de Aplicación	Características	Algoritmo Sugerido	Justificación
Sistema embebido	Memoria limitada, predecibilidad	Heap Sort	$O(1)$ espacio, $O(n \log n)$ garantizado
Base de datos	Grandes volúmenes, estabilidad	Merge Sort	Estable, comportamiento predecible
Juegos en tiempo real	Datos pequeños, velocidad	Inserción	Muy rápido para n pequeño
Algoritmo genérico	Versatilidad, buen promedio	Quick Sort	Excelente caso promedio
Sistema crítico	Tiempo acotado, sin variación	Heap Sort	Sin peor caso inesperado
Prototipo/Learning	Comprensión, simplicidad	Selección o Burbuja	Fácil de entender e implementar

Ejemplo

$$f(n) = n^2 + 5n + 25$$

$$f(n) \leq c \cdot g(n) \text{ para todo } n \geq n_0$$

$$\text{Para } n = n_0$$

$$c n_0^2 = n_0^2 + 5 n_0 + 25$$

$$c = 1 + 5/n_0 + 25/n_0^2$$

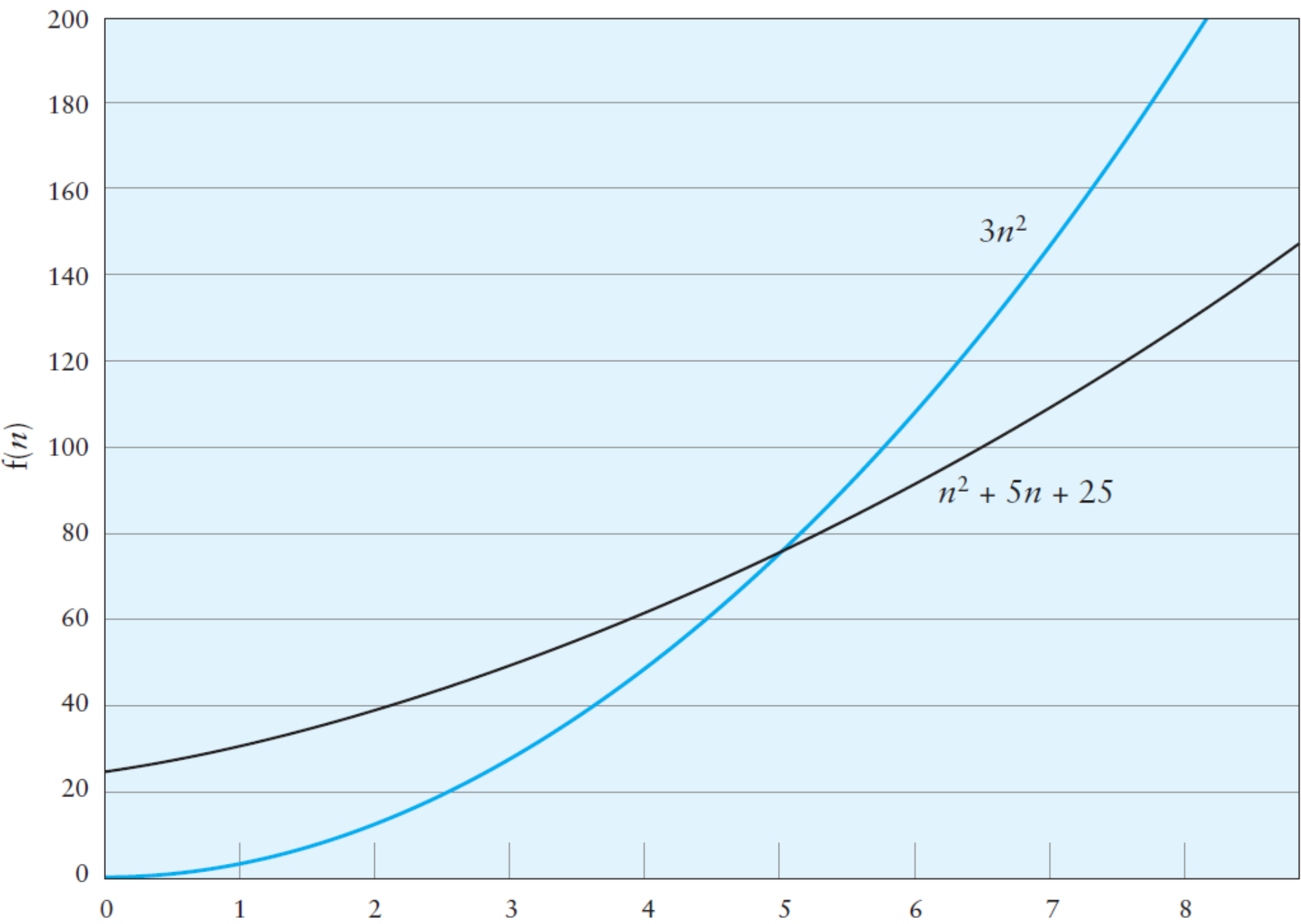
$$\text{Para } n_0 = 5 \Rightarrow c = 3$$

$$n^2 + 5n + 25 \leq 3n^2 \text{ para todo } n \geq n_0$$

Formal Definition of Big-O

Consider a program that is structured as follows:

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        Simple Statement  
    }  
}  
for (int k = 0; k < n; k++) {  
    Simple Statement 1  
    Simple Statement 2  
    Simple Statement 3  
    Simple Statement 4  
    Simple Statement 5  
}  
Simple Statement 6  
Simple Statement 7  
...  
Simple Statement 30
```



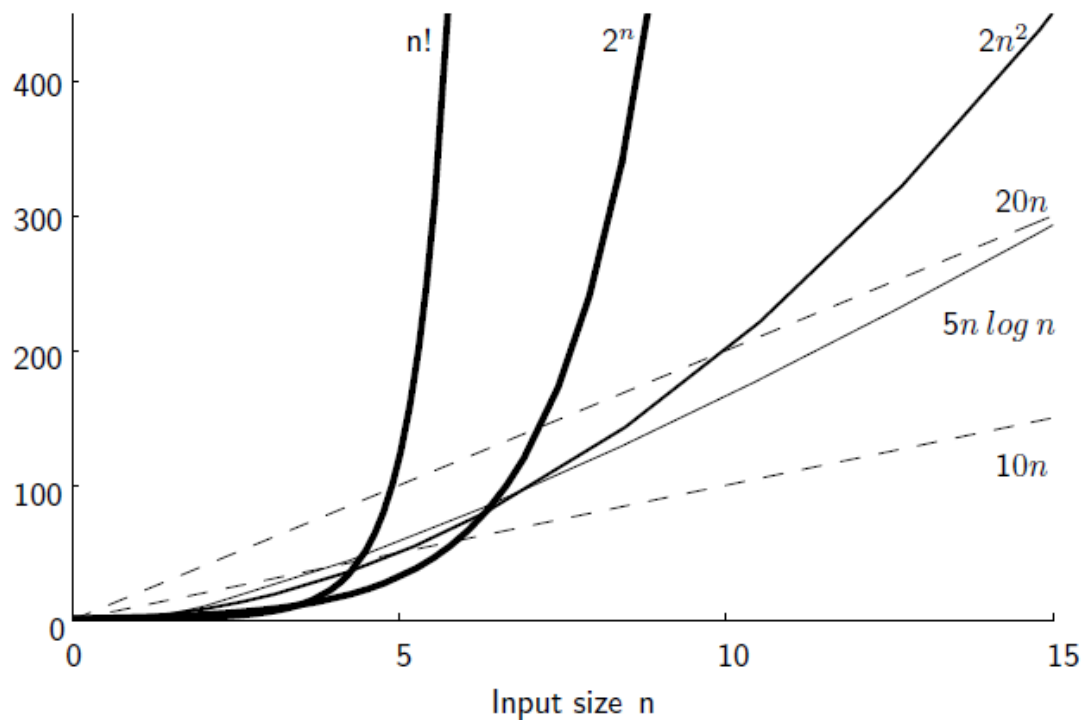
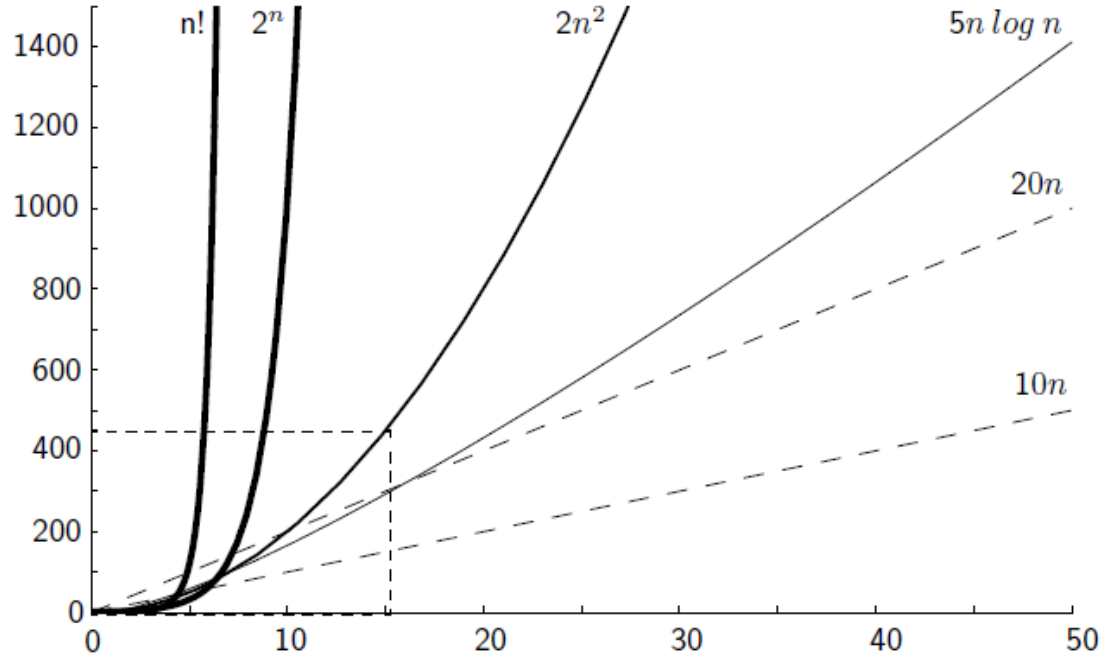


TABLE 2.6

Effects of Different Growth Rates

$O(f(n))$	$f(50)$	$f(100)$	$f(100)/f(50)$
$O(1)$	1	1	1
$O(\log n)$	5.64	6.64	1.18
$O(n)$	50	100	2
$O(n \log n)$	282	664	2.35
$O(n^2)$	2500	10,000	4
$O(n^3)$	12,500	100,000	8
$O(2^n)$	1.126×10^{15}	1.27×10^{30}	1.126×10^{15}
$O(n!)$	3.0×10^{64}	9.3×10^{157}	3.1×10^{93}

Big O

Algorithm Analysis

- (1)

```
sum = 0;
for( i = 0; i < n; ++i )
    ++sum;
```
- (2)

```
sum = 0;
for( i = 0; i < n; ++i )
    for( j = 0; j < n; ++j )
        ++sum;
```
- (3)

```
sum = 0;
for( i = 0; i < n; ++i )
    for( j = 0; j < n * n; ++j )
        ++sum;
```
- (4)

```
sum = 0;
for( i = 0; i < n; ++i )
    for( j = 0; j < i; ++j )
        ++sum;
```

Análisis Asintótico

Errores Comunes y Malentendidos:

Error 1: Confundir las notaciones

Incorrecto: "Este algoritmo es $O(n)$ en el mejor caso"

Correcto: "Este algoritmo es $\Omega(n)$ en el mejor caso"

Error 2: Usar Big O para todo

Incorrecto: "La búsqueda binaria es $O(\log n)$ "

Más preciso: "La búsqueda binaria es $\Theta(\log n)$ "

Error 3: Incluir constantes

Incorrecto: $f(n) = O(5n^2)$

Correcto: $f(n) = O(n^2)$

Error 4: No considerar el dominio

Las notaciones asintóticas se aplican cuando n tiende a infinito.

Análisis Asintótico

Usar **Big O** cuando:

- Queremos garantizar un límite superior
- Analizamos el peor caso del algoritmo
- Necesitamos asegurar que el algoritmo no será peor que cierto límite

Usar **Big Omega Ω** cuando:

- Queremos establecer un límite inferior
- Analizamos el mejor caso del algoritmo
- Necesitamos demostrar que un problema requiere al menos cierto tiempo

Usar **Big Theta Θ** cuando:

- El mejor y peor caso tienen el mismo orden de crecimiento
- Queremos dar la caracterización más precisa
- El algoritmo tiene comportamiento consistente independiente de la entrada

Análisis Asintótico

El análisis asintótico es una forma de estimación rápida del consumo de recursos de un algoritmo.

Proporciona un modelo simplificado del tiempo de ejecución u otras necesidades de recursos de un algoritmo.

Esta simplificación suele ayudar a comprender el comportamiento de los algoritmos.

UNC
Fac. Ciencias Exactas
Físicas y Naturales

AED – Unidad 5
Ordenamiento

Ordenamiento (sort)

$O(n^2)$

Bubble Sort

FIGURE 10.1
One Pass of Bubble Sort

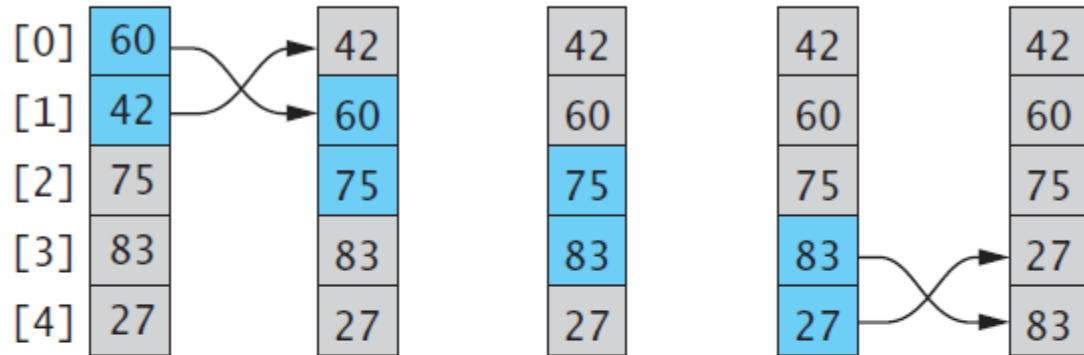
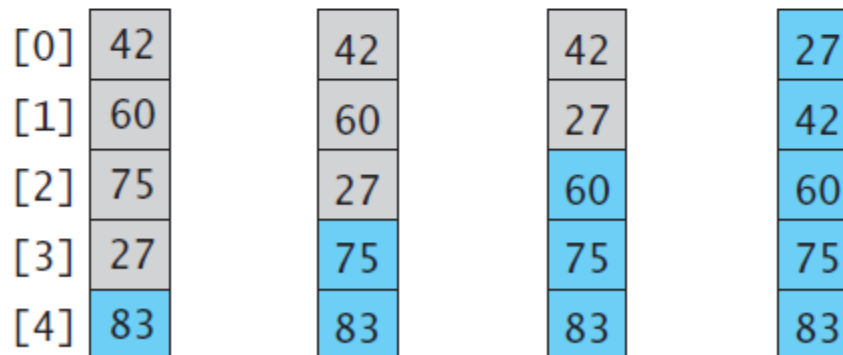


FIGURE 10.2
Array After Completion
of Each Pass



Bubble Sort

El ordenamiento de burbuja compara elementos adyacentes de un arreglo e intercambia sus valores si están desordenados. De esta manera, los valores más pequeños se desplazan hacia la parte superior del arreglo (hacia el primer elemento), mientras que los valores más grandes se desplazan hacia la parte inferior; de ahí su nombre.

	Comparaciones		Intercambios	
	Best	Worst	Best	Worst
Bubble	$O(n)$	$O(n^2)$	$O(1)$	$O(n^2)$

Bubble Sort Algorithm

1. **do**
2. **for** each pair of adjacent array elements
3. **if** the values in a pair are out of order
4. Exchange the values
5. **while** the array is not sorted.

Selection Sort

	i=0	1	2	3	4	5	6
42	<u>13</u>	13	13	13	13	13	13
20	20	14	14	14	14	14	14
17	17	<u>17</u>	15	15	15	15	15
13	42	42	<u>42</u>	17	17	17	17
28	28	28	28	<u>28</u>	20	20	20
14	14	20	20	20	<u>28</u>	23	23
23	23	23	23	23	23	<u>28</u>	28
15	15	15	17	42	42	42	<u>42</u>

Selection Sort

La ordenación por selección es un algoritmo que ordena un arreglo (o secuencia) realizando varias pasadas a través de ella, seleccionando el siguiente menor cada vez y colocándolo en su posición final en el arreglo.

	Comparaciones		Intercambios	
	Best	Worst	Best	Worst
Selection	$O(n^2)$	$O(n^2)$	$O(n)$	$O(n)$

Selection Sort Algorithm

1. **for** `fill = 0` to `n - 2` **do**
2. Set `pos_min` to the subscript of the smallest item in the subarray starting at subscript `fill`.
3. **if** the next smallest item is not at position `fill`
 Exchange the item at `pos_min` with the one at `fill`.

Insertion Sort



30
25
15
20
28

25
30
15
20
28

15
25
30
20
28

15
20
25
30
28

15
20
25
28
30

End of
pass 1

End of
pass 2

End of
pass 3

End of
pass 4

Insertion Sort

La ordenación por inserción itera sobre una lista de elementos. Cada elemento se inserta, uno por uno, en la posición correcta dentro de una lista ordenada compuesta por los elementos ya procesados.

	Comparaciones		Intercambios	
	Best	Worst	Best	Worst
Insertion	$O(n)$	$O(n^2)$	$O(n)$	$O(n^2)$

Insertion Sort Algorithm

1. **for** each array element from the second element to the last element
2. Insert the selected element where it belongs in the array, increasing the length of the sorted subarray by 1 element.

To accomplish Step 2, the insertion step, we need to make room for the element to be inserted (saved in `next_val`) by shifting all values that are larger than it, starting with the last value in the sorted subarray.

Comparaciones versus Intercambios

Hemos analizado las comparaciones y los intercambios por separado, cual es más costoso (en términos de tiempo de procesamiento)?.

En C++, un intercambio requiere que la computadora intercambie dos elementos utilizando un tercero como intermediario. Una comparación requiere que la computadora evalúe el operador $<$ o llame a una función de comparación. El costo de ejecutar una función de comparación depende de su complejidad, pero probablemente será mayor que el de un intercambio debido a la sobrecarga para llamar y ejecutar la función. Un intercambio requiere mover físicamente la información de cada elemento. El costo de un intercambio es proporcional al tamaño de los elementos intercambiados y, por lo tanto, puede ser más costoso que una comparación para elementos grandes.

Comparison of Quadratic Sorts

	Number of Comparisons		Number of Exchanges	
	Best	Worst	Best	Worst
Selection sort	$O(n^2)$	$O(n^2)$	$O(n)$	$O(n)$
Bubble sort	$O(n)$	$O(n^2)$	$O(1)$	$O(n^2)$
Insertion sort	$O(n)$	$O(n^2)$	$O(n)$	$O(n^2)$

Shell Sort

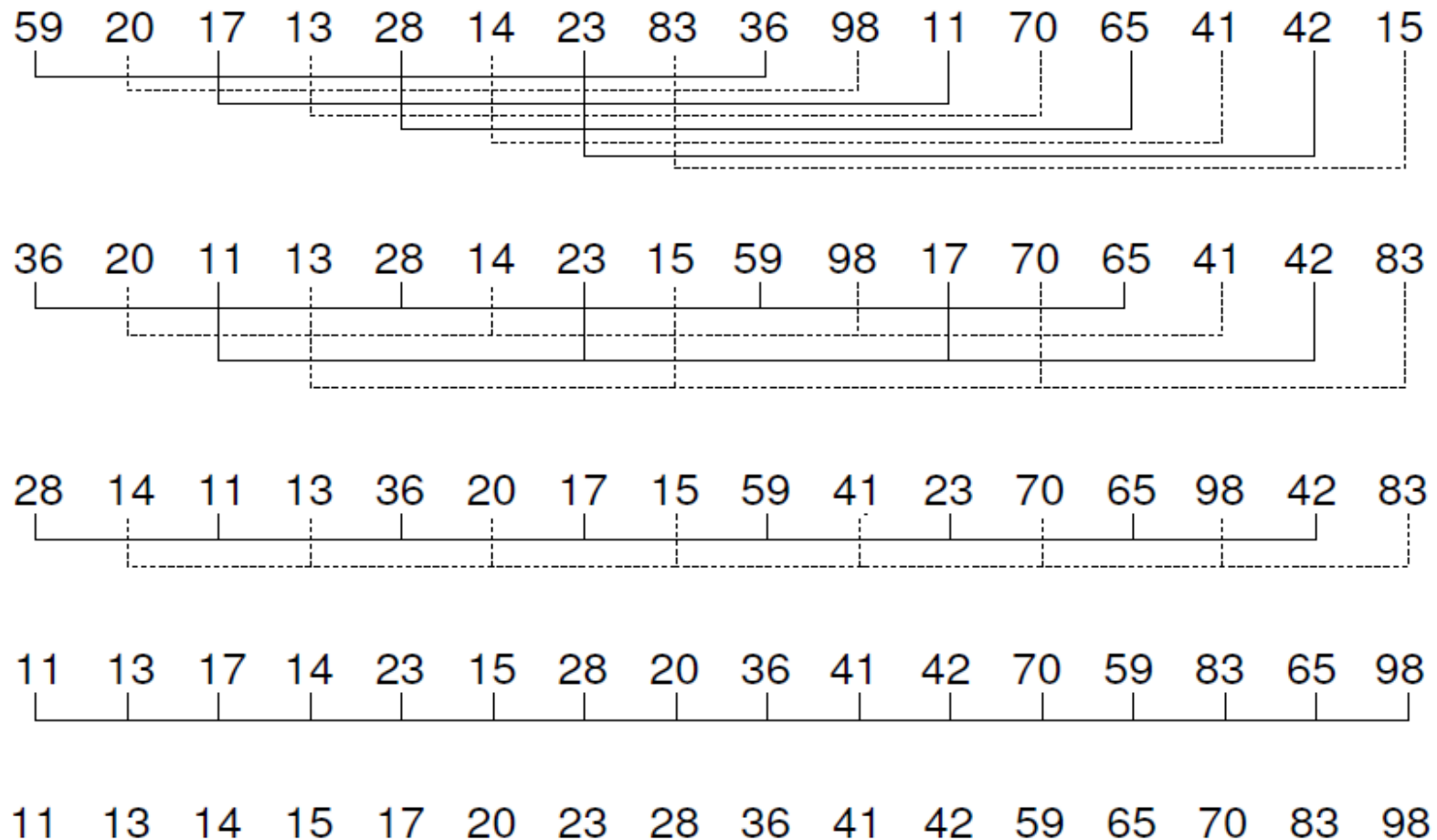


Figure 7.6 An example of Shellsort. Sixteen items are sorted in four passes. The first pass sorts 8 sublists of size 2 and increment 8. The second pass sorts 4 sublists of size 4 and increment 4. The third pass sorts 2 sublists of size 8 and increment 2. The fourth pass sorts 1 list of size 16 and increment 1 (a regular Insertion Sort).

Shell Sort: se puede considerar el ordenamiento de Shell como un enfoque de "divide y vencerás" para el ordenamiento por inserción. En lugar de ordenar todo el array al principio, la idea del ordenamiento de Shell es ordenar subarrays más pequeños mediante el ordenamiento por **inserción** antes de ordenar todo el array. Los subarrays iniciales contendrán dos o tres elementos, por lo que el ordenamiento por inserción será muy rápido. Tras ordenar cada conjunto de subarrays, se ordenará un nuevo conjunto con aproximadamente el doble de elementos. El último paso consiste en realizar un ordenamiento por inserción en todo el array, que ha sido preordenado por los ordenamientos anteriores.

Shell Sort Algorithm

1. Set the initial value of gap to $n/2$.
2. **while** gap > 0
3. **for** each array element from position gap to the last element
4. Insert this element where it belongs in its subarray.
5. **if** gap is 2, set it to 1.
6. **else** gap = gap/2.

<https://www.youtube.com/watch?v=qzXAVXddcPU>

El análisis general del ordenamiento de Shell es un **problema** de investigación **abierto** en informática. El rendimiento depende de cómo se elija la secuencia decreciente de valores para **gap**.

Se sabe que si se utilizan potencias sucesivas de 2 para **gap** (es decir, 32, 16, 8, 4, 2, 1) el ordenamiento de Shell es $O(n^2)$. Sin embargo, si los valores sucesivos para **gap** tienen la forma $2^k - 1$ (es decir, 31, 15, 7, 3, 1), se puede demostrar que el rendimiento es $O(n^{3/2})$. Esta secuencia se conoce como secuencia de Hibbard. Existen otras secuencias con un rendimiento similar o superior. Por ejemplo si el valor inicial de **gap** es $n/2$ y luego dividimos por 2.2 truncando al siguiente entero más bajo. Los estudios empíricos de este enfoque muestran que el rendimiento es $O(n^{5/4})$ o incluso $O(n^{7/6})$, pero no existe una base teórica para este resultado.

Shell Sort Algorithm

1. Set the initial value of gap to $n/2$.
2. **while** gap > 0
3. **for** each array element from position gap to the last element
4. Insert this element where it belongs in its subarray.
5. **if** gap is 2, set it to 1.
6. **else** gap = gap/2.2.

UNC
Fac. Ciencias Exactas
Físicas y Naturales

AED – Árboles

Árbol

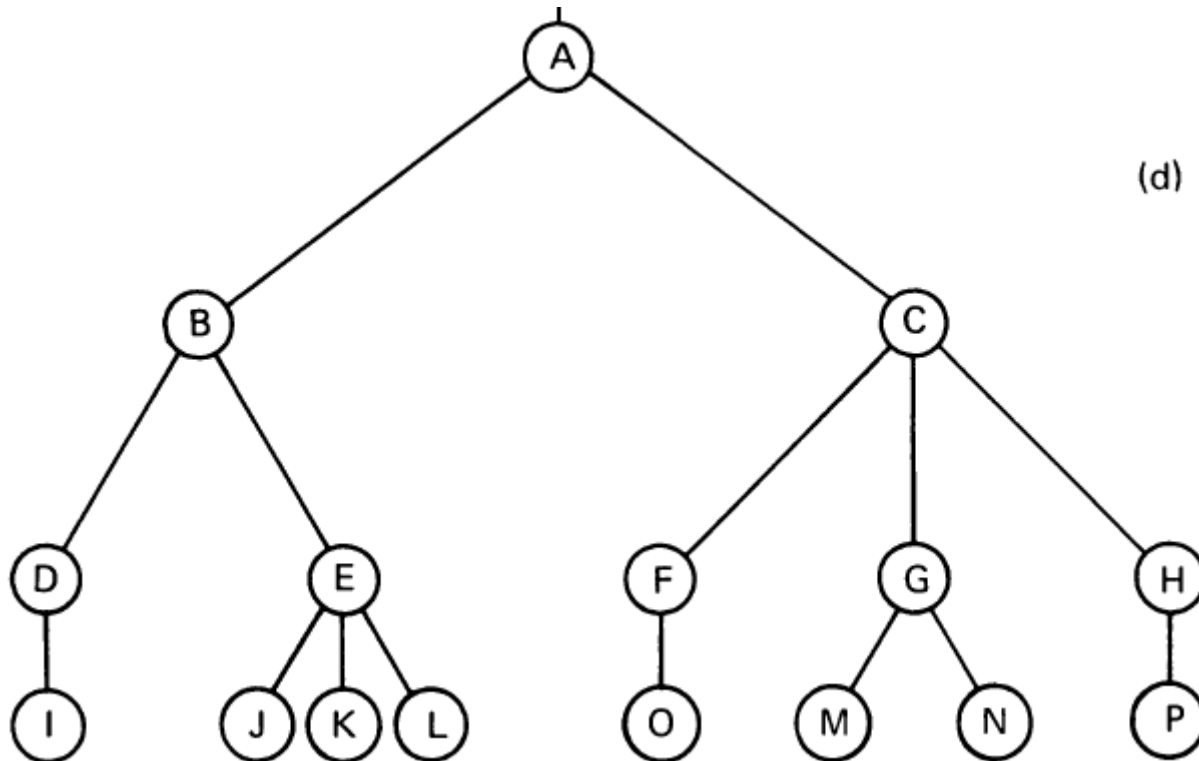
Es una estructura de datos jerárquica (no lineal)

Basada en nodos que almacena distintos tipos de datos (Por ej: Árbol de enteros)

Componentes

Nodo, Raíz, Padre, Hijo, Hermano, Hoja, Arista,

Subárbol, Camino, Longitud de un camino, Profundidad de un Nodo, Altura del árbol



Árbol

Es una estructura de datos jerárquica (no lineal)

Basada en nodos que almacena distintos tipos de datos (Por ej: Árbol de enteros)

Componentes

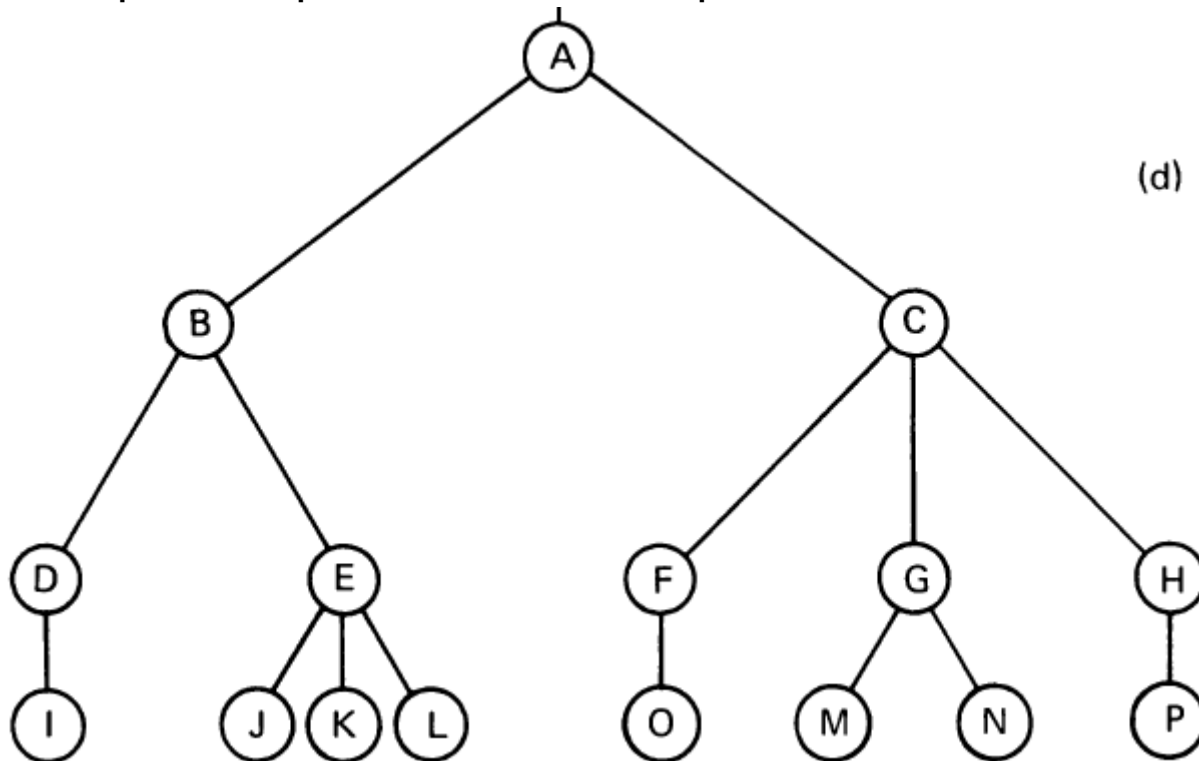
Nodo: almacena la información (+ referencias a sus hijos si es dinámico)

Raíz: único Nodo que no tiene padre es el Ancestro de todos

Padre: nodo inmediatamente superior a otro nodo

Hijo: nodo que tiene un padre

Hermanos: nodos que comparten un mismo padre



Árbol

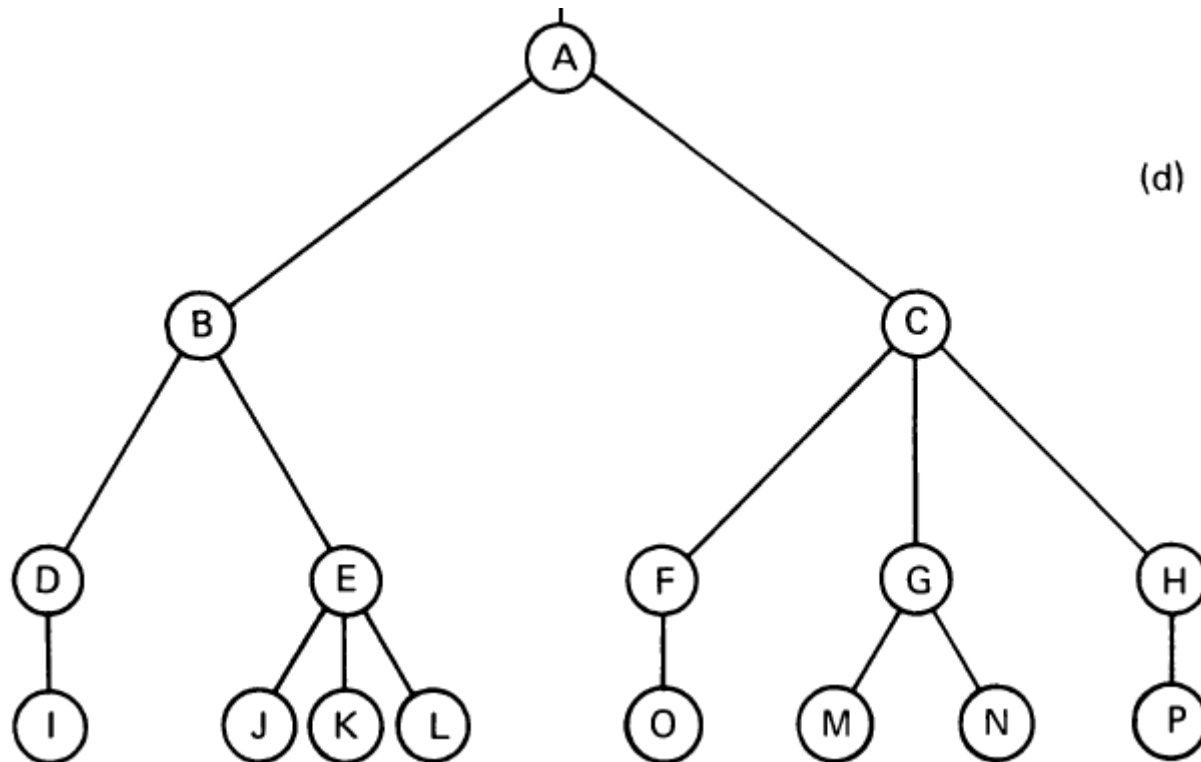
Hoja: nodo que no tiene hijos, o nodo terminal

Arista: conexión o enlace entre dos nodos

Subárbol: Un subárbol de un árbol T es un árbol S cuya raíz es algún nodo n de T, y S contiene a n y a todos los descendientes de n en T

Camino: secuencia de nodos donde cada nodo consecutivo (en la secuencia) está conectado al siguiente por una arista

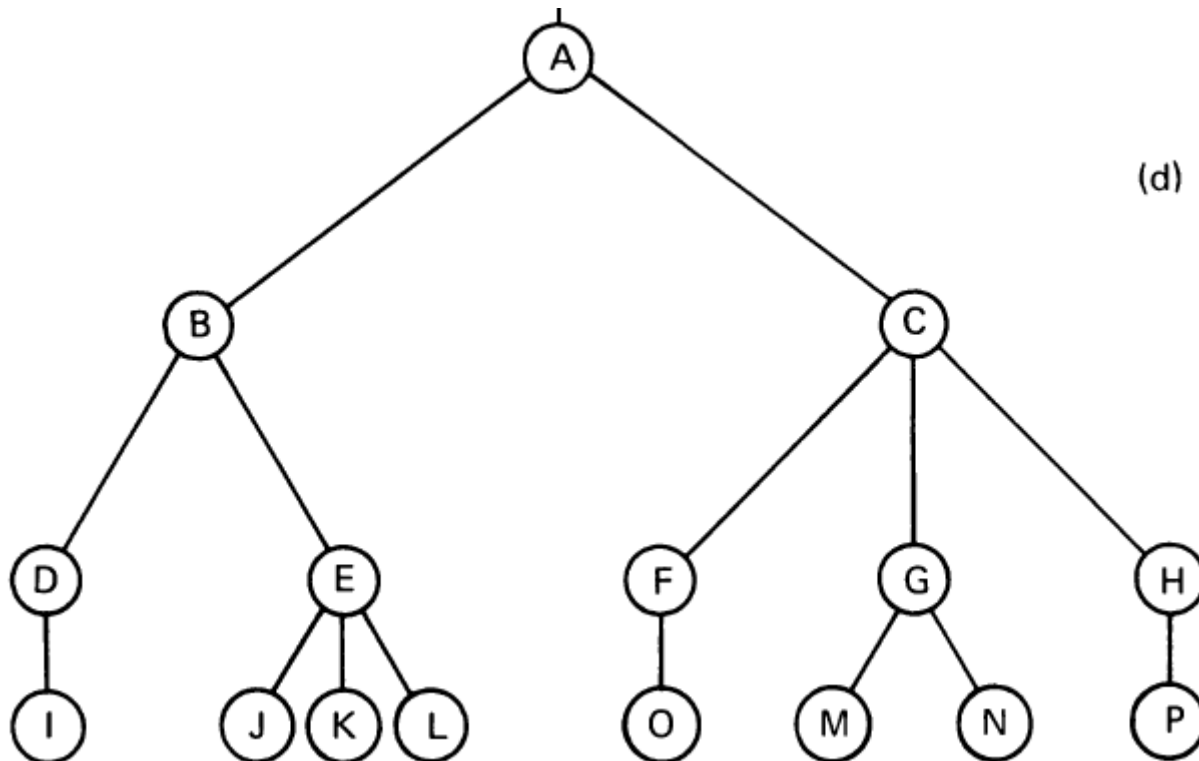
Longitud de un camino: número de aristas de un Camino



Árbol

Profundidad de un Nodo: La profundidad de un nodo n en un árbol T es la longitud del único camino desde la raíz de T hasta n . Si la raíz es el nodo n , su profundidad es 0

Altura del árbol: el camino más largo desde la raíz hasta una hoja, **OJO** se puede contar como aristas (raíz 0) o nodos (raíz 1)



Tipos de Árboles

Árbol General: un árbol donde cada nodo puede tener un número arbitrario de hijos (ej: organigrama de una empresa)



Tipos de Árboles

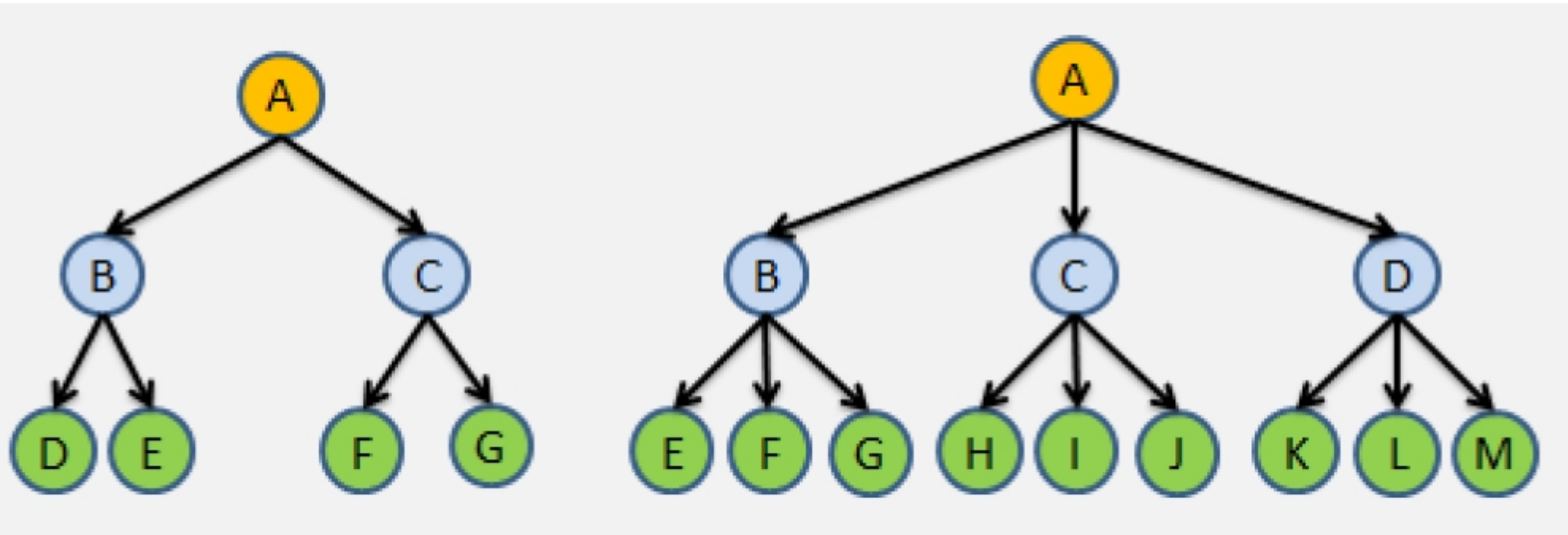
Árboles m-arios: cada nodo se vincula con **m** hijos

Si $m=2$ -> **árbol binario**

Si $m=3$ -> **árbol ternario**

Si $m=4$ -> **árbol cuaternario**

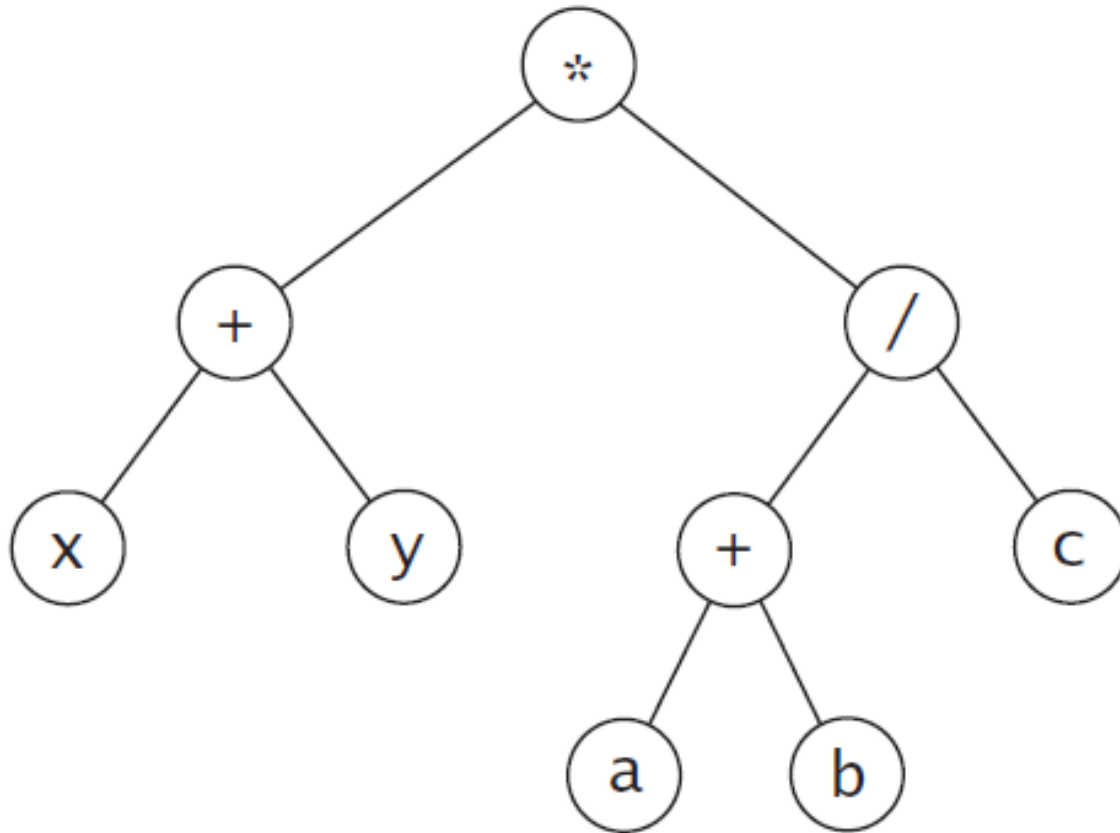
En general trabajaremos con árboles binarios salvo que se especifique explícitamente lo contrario



Árboles binarios

Algunos de los más comunes en computer science

Árbol que almacena una expresión aritmética, cada nodo almacena un operador o un operando. Ejemplo : $(x+y)*((a+b)/c)$



Árboles binarios

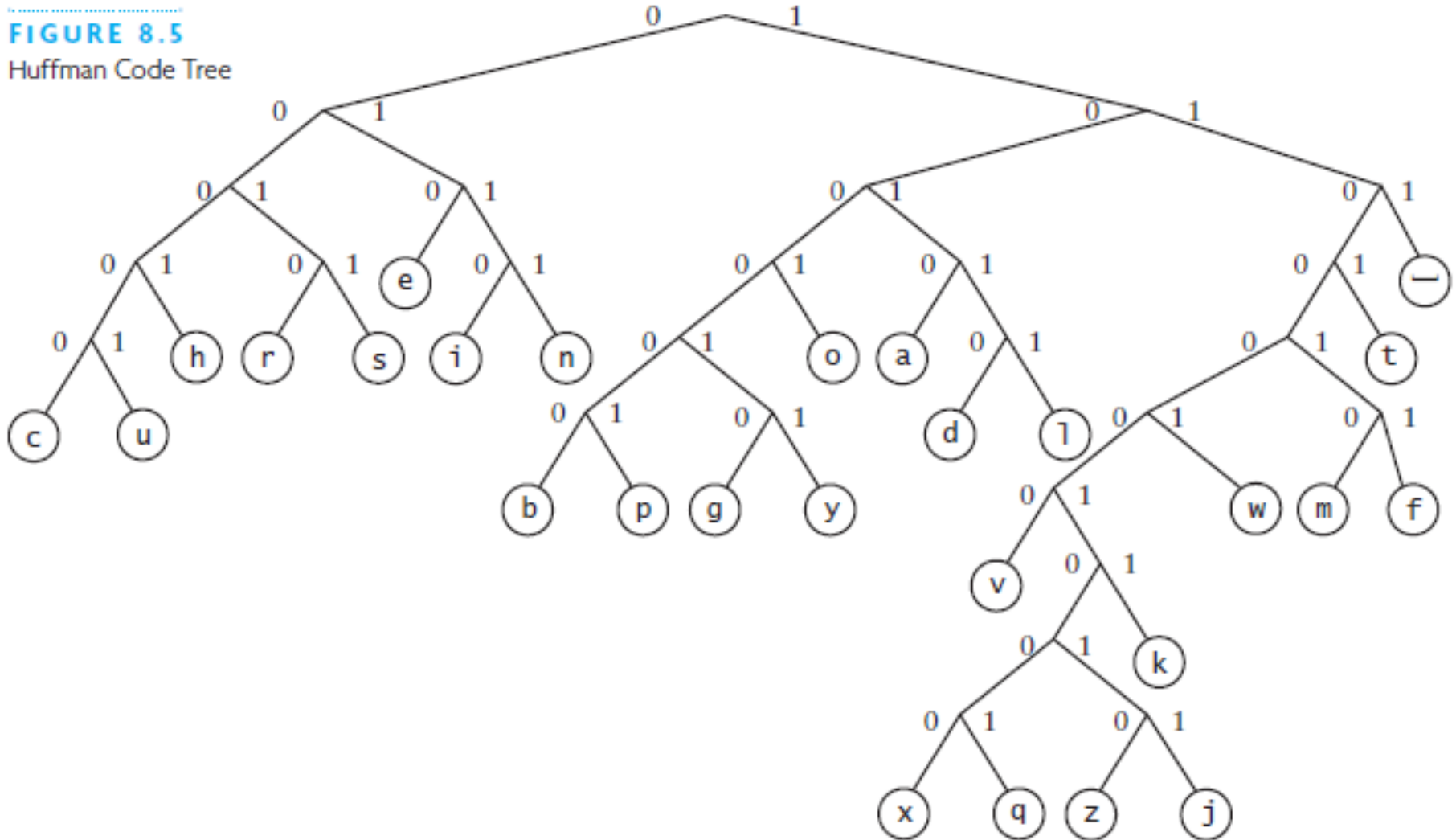
Algunos de los más comunes en computer science

Árbol de Huffman: se utiliza para la compresión de archivos de texto

Utiliza menos bits para las letras más comunes (por ejemplo, espacio, e, a y t) y más bits para las menos comunes (por ejemplo, q, x y z).

Ejemplo: la 'd' es el 10110, el ' ' es el 111

FIGURE 8.5
Huffman Code Tree



Árboles binarios

Algunos de los más comunes en computer science

Árbol Binario de Búsqueda (ABB o BST)

En un abb se cumple que

- $\text{raiz}(\text{abb}) > \text{raíz}(\text{hi}(\text{abb}))$
- $\text{raiz}(\text{abb}) < \text{raíz}(\text{hd}(\text{abb}))$
- $\text{hi}(\text{abb})$ también es abb
- $\text{hd}(\text{abb})$ también es abb

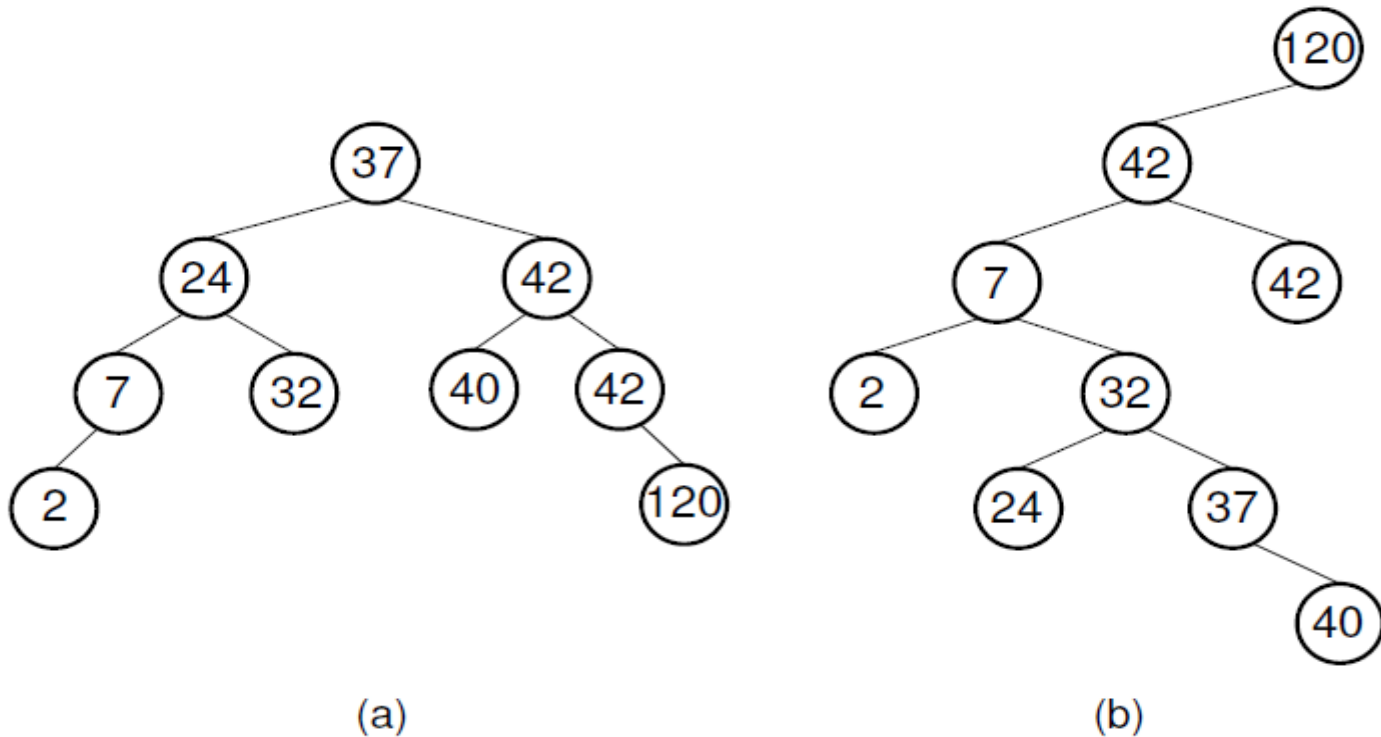
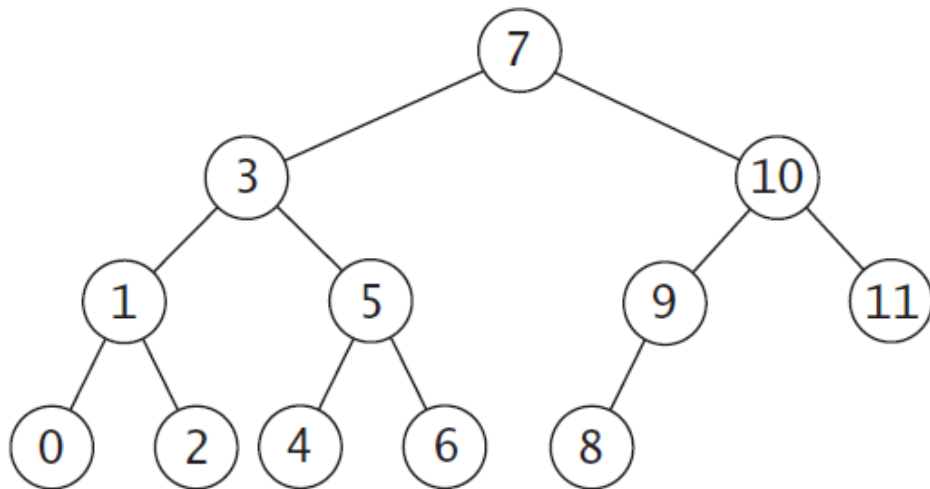
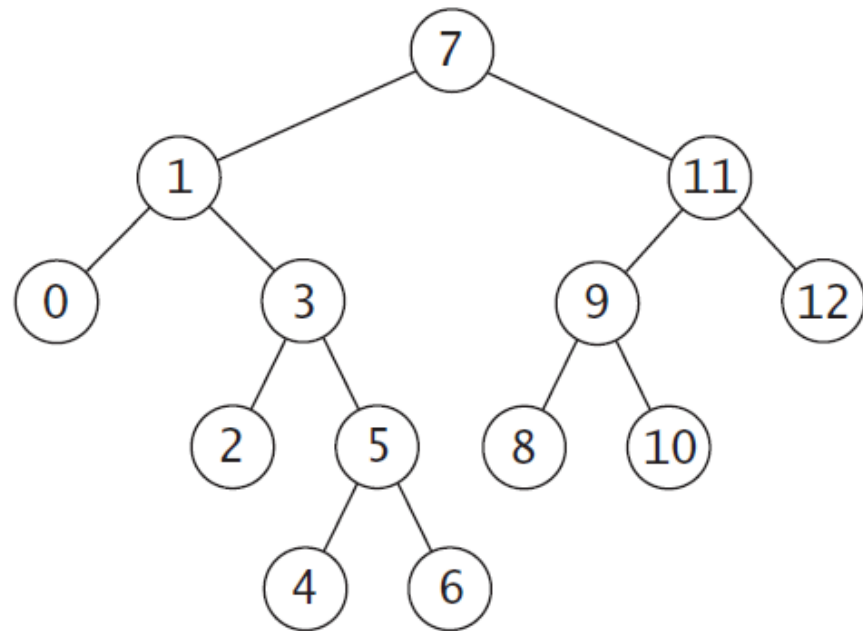


Figure 5.13 Two Binary Search Trees for a collection of values. Tree (a) results if values are inserted in the order 37, 24, 42, 7, 2, 40, 42, 32, 120. Tree (b) results if the same values are inserted in the order 120, 42, 42, 7, 2, 32, 37, 24, 40.

Árboles binarios búsqueda

Árbol binario lleno: **cada nodo tiene exactamente cero o dos hijos**. En otras palabras, ningún nodo en un árbol binario lleno tiene solo un hijo

Árbol binario completo: **completo** si todos los niveles, **excepto posiblemente el último**, están completamente llenos, y todos los nodos en el último nivel están lo más a la izquierda posible



Árboles binarios

Recorrido de un árbol:

Preorden: visito la R, recorro hi, recorro hd

Inorden: recorro hi, visito la R, recorro hd

Posorden: recorro hi, recorro hd, visito la R

1. *Preorder* : R, A, B (visit root *before* the subtrees)
2. *Inorder* : A, R, B
3. *Postorder*: A, B, R (visit root *after* the subtrees)

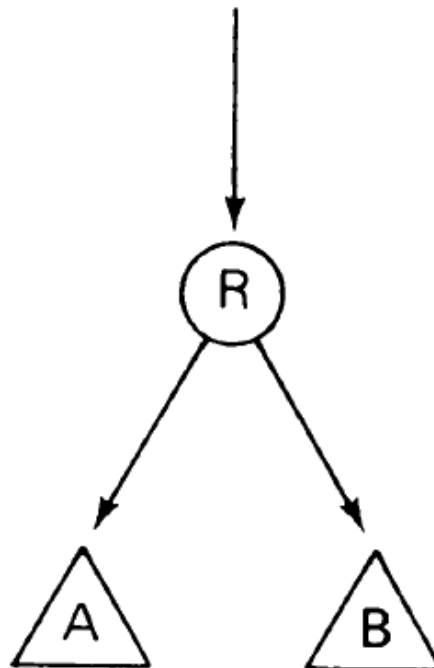


Fig. 4.24 Binary tree.

Árboles binarios búsqueda

Borrado en ABB

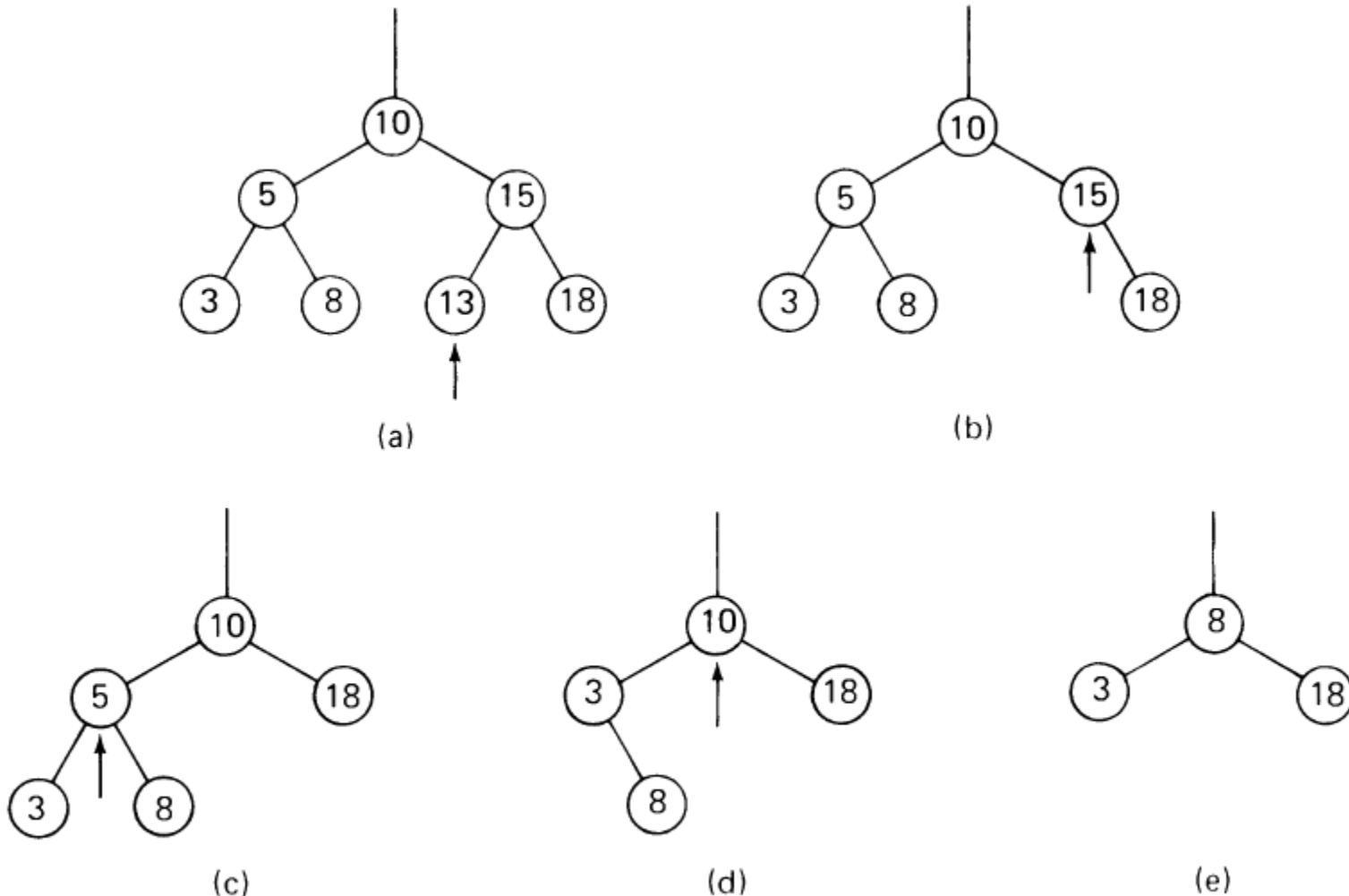


Fig. 4.28 Tree deletion.

Especificación algebraica para el Tipo de Dato Abstracto (TDA) ABB

1_Signatura

Tipos: árbol T, elementos del árbol A

Operaciones:

crearArbol() -> T (crea ABB vacío)

insertar(A,T)-> T (agrega elemento al árbol T)

raiz(T) -> A (devuelve la raíz de T)

buscar(A,T) -> booleano

eliminar(A,T) -> T (devuelve T sin el elemento A)

esVacio(T) -> booleano

2_Axiomas (reglas que rigen el comportamiento de las operaciones)

un árbol recién creado está vacío esVacio(crearArbol()) -> true

un árbol con elementos no está vacío esVacio(insertar(A,T)) -> false

buscar(A,crearArbol())-> false

buscar(A,insertar(A,T))-> true

eliminar(A,insertar(A,T)) -> T

ABB implementado dinámicamente con nodos enlazados

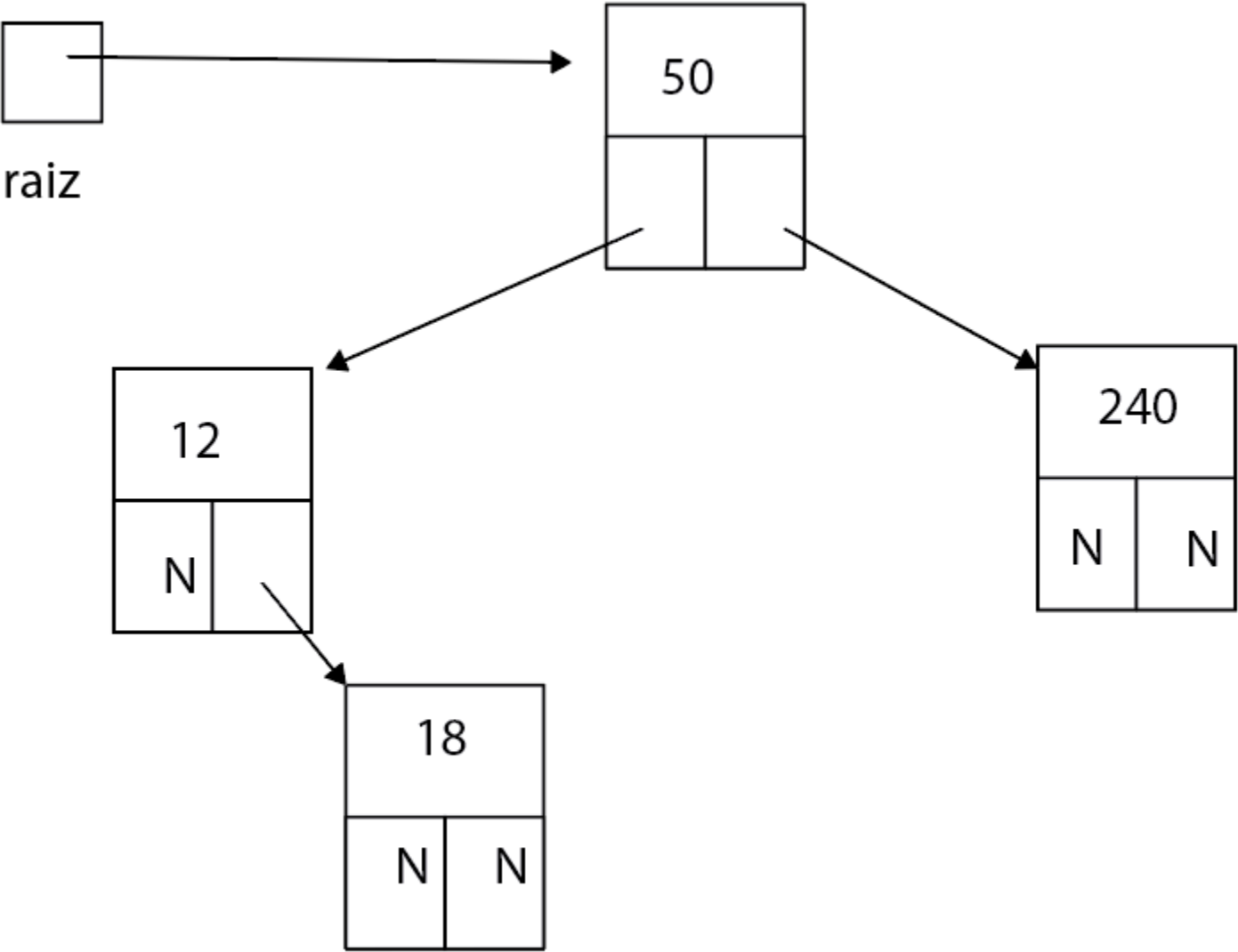


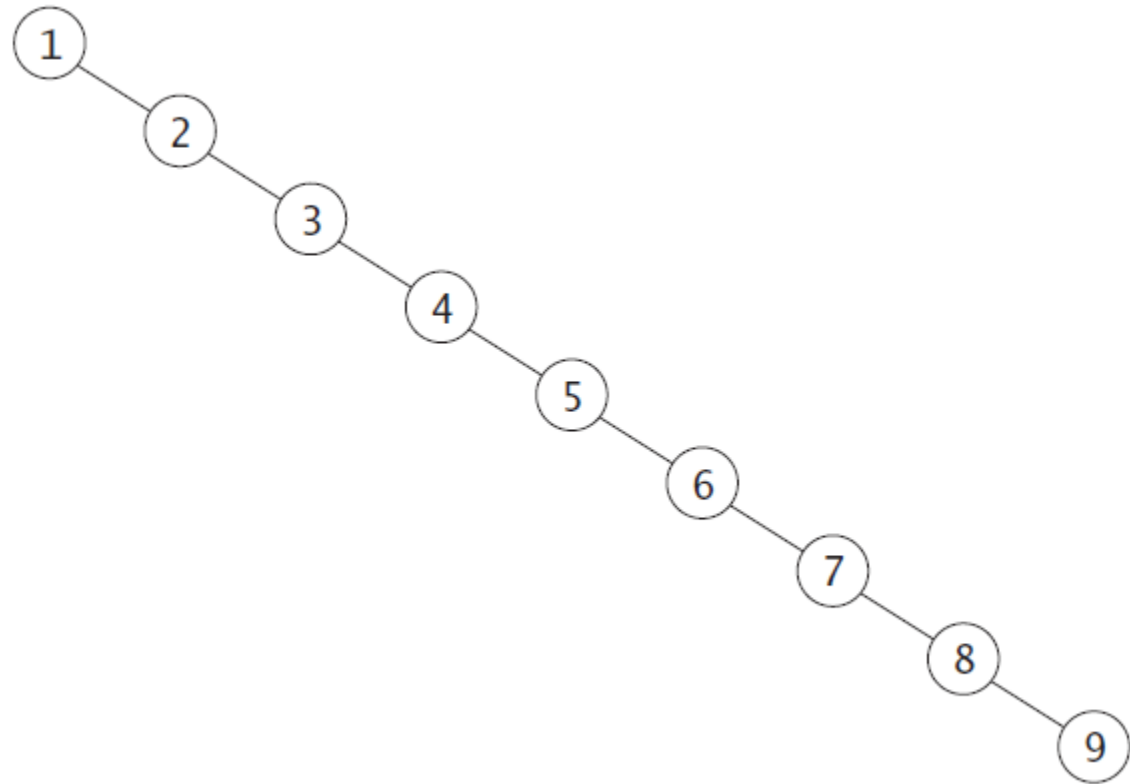
ABB implementado dinámicamente con nodos enlazados
Veamos el código C++

Árboles Binarios

Árbol Binario de Búsqueda (ABB – BST)

FIGURE 11.1

Very Unbalanced Binary
Search Tree



Árbol AVL (Adelson-Velskii y Landis)

Definición de altura de un árbol (T)

$H(T)$ es 0 si T solo contiene la raíz

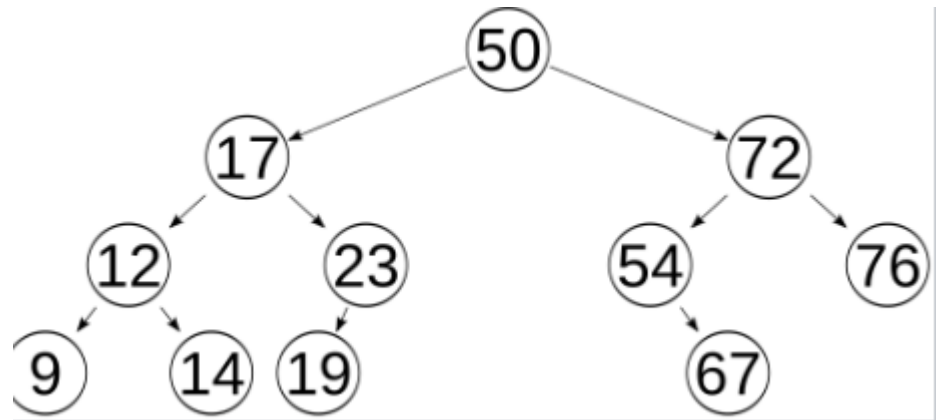
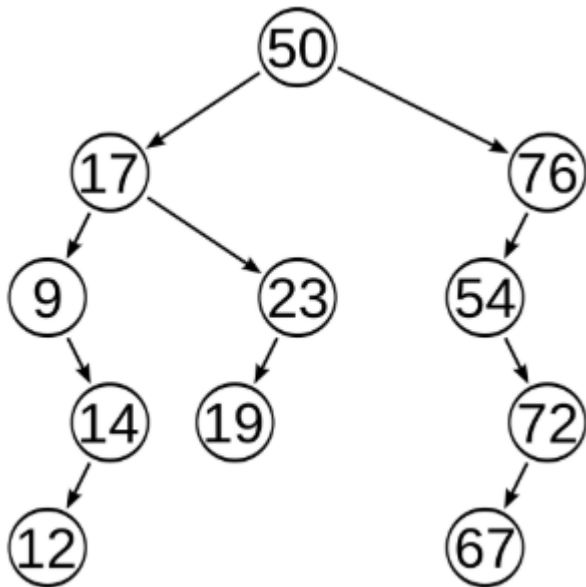
$$1 + \max(H(T_i), H(T_d))$$

Definición de árbol AVL

Un árbol vacío es un árbol AVL

Si T es un árbol no vacío y T_i y T_d sus subárboles, entonces T es AVL si:

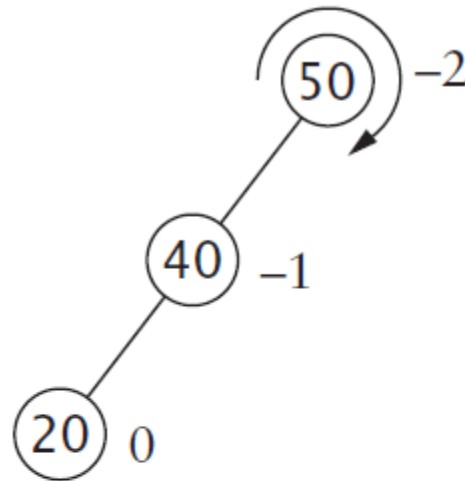
- T_i es AVL
- T_d es AVL
- $|H(T_i) - H(T_d)| \leq 1$



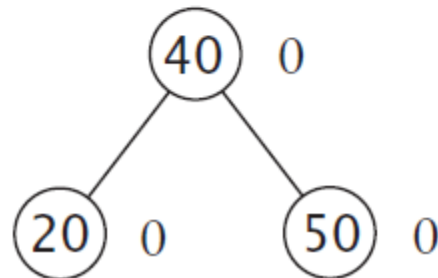
Árbol AVL (Adelson-Velskii y Landis)

Tipos de rotaciones:

Izq-Izq: padre tiene un desbalance de -2, hijo izq desbalance de -1 >> Rotar a la derecha alrededor del padre



If we rotate the tree right around the root tree is also a binary search tree, but all no

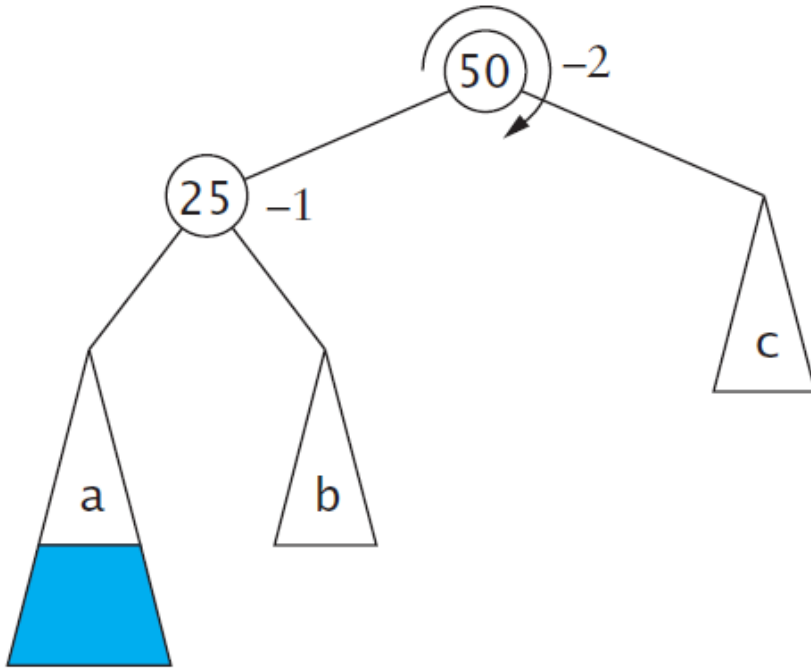


Árbol AVL (Adelson-Velskii y Landis)

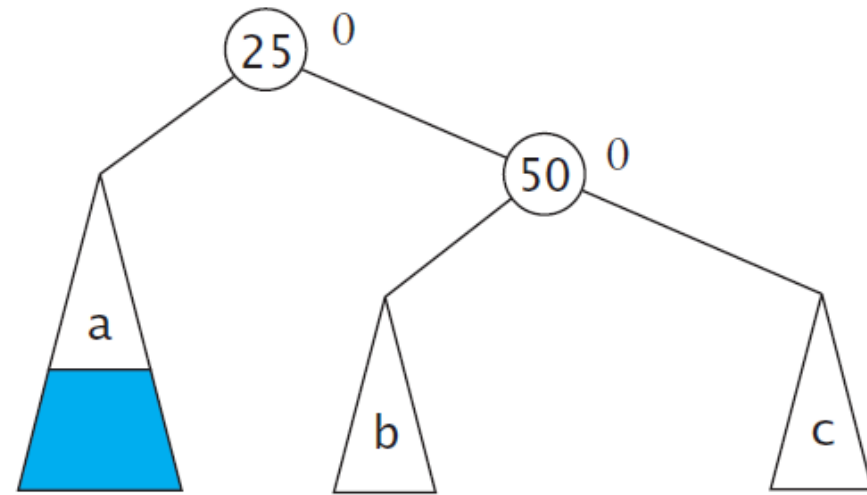
Tipos de rotaciones:

Izq-Izq: padre tiene un desbalance de -2, hijo izq desbalance de -1 >> Rotar a la derecha alrededor del padre

Left-Heavy Tree



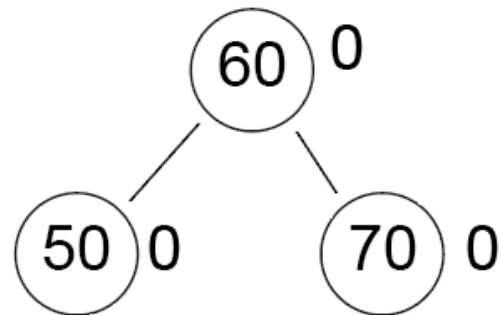
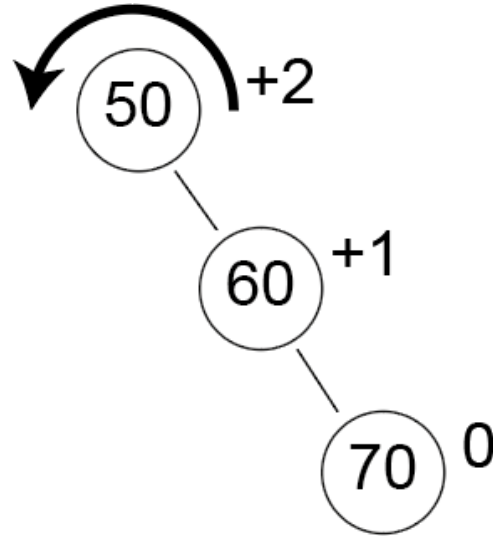
Left-Heavy Tree After Rotation Right



Árbol AVL (Adelson-Velskii y Landis)

Tipos de rotaciones:

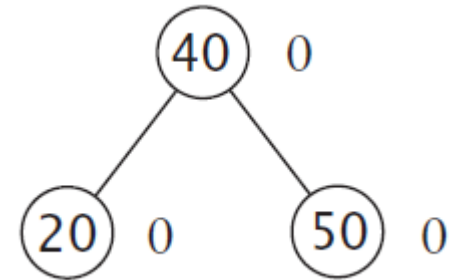
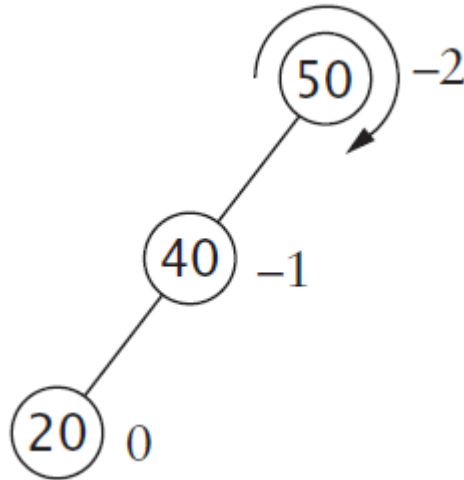
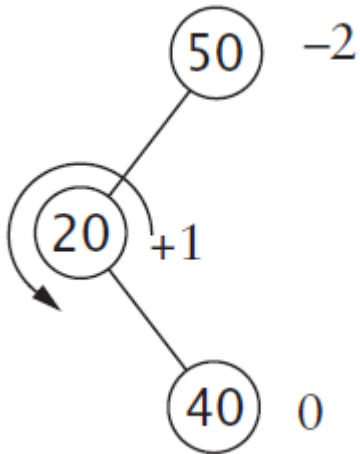
Der-Der: padre tiene un desbalance de +2, hijo der desbalance de +1>>
Rotar a la izquierda alrededor del padre



Árbol AVL (Adelson-Velskii y Landis)

Tipos de rotaciones:

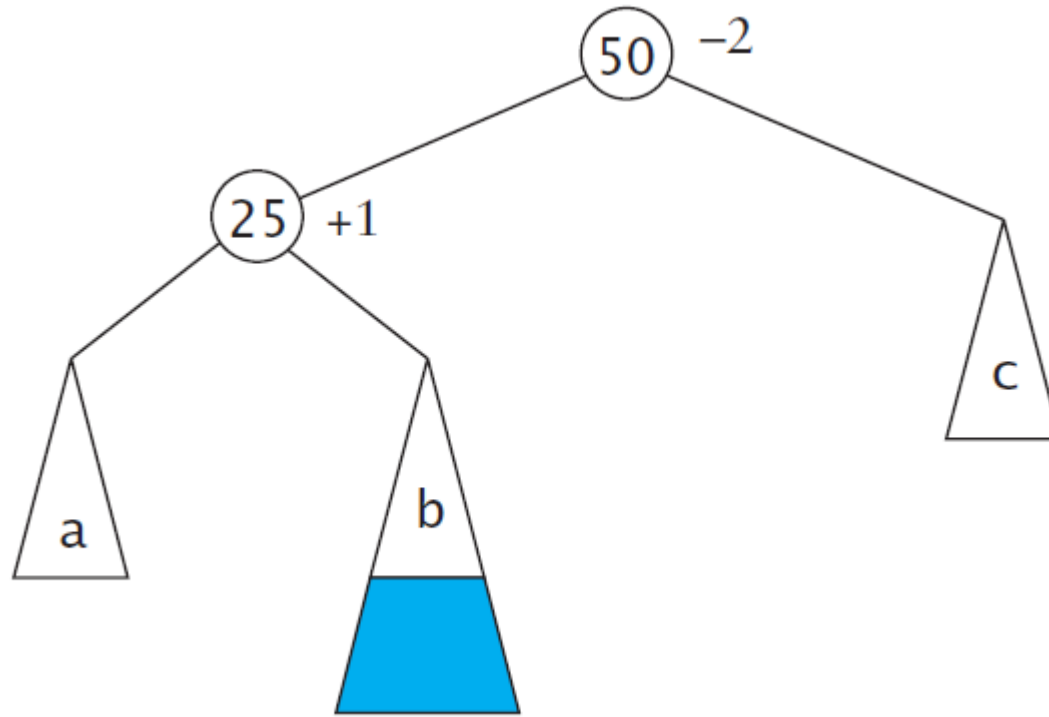
Izq-Der: padre tiene un desbalance de -2, hijo izq desbalance de +1 >> Rotar a la izq alrededor del hijo, luego rotar a la derecha alrededor del padre



Árbol AVL (Adelson-Velskii y Landis)

Tipos de rotaciones:

Izq-Der: padre tiene un desbalance de -2, hijo izq desbalance de +1 >> Rotar a la izq alrededor del hijo, luego rotar a la derecha alrededor del padre

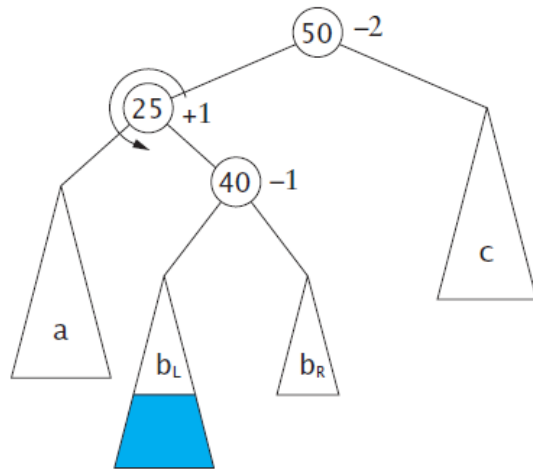


Árbol AVL (Adelson-Velskii y Landis)

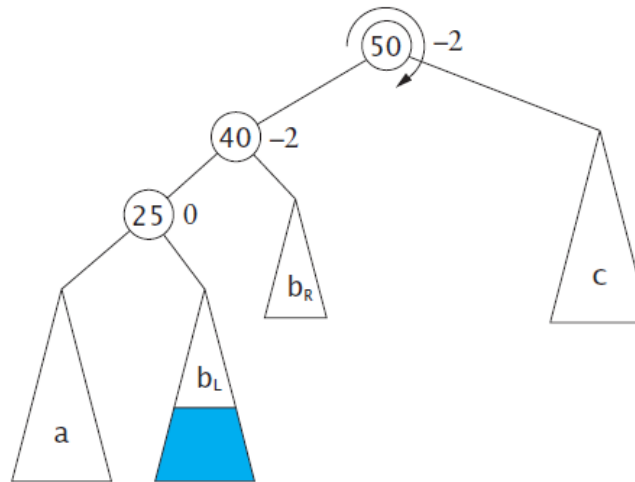
Tipos de rotaciones:

Izq-Der: padre tiene un desbalance de -2, hijo izq desbalance de +1 >> Rotar a la izq alrededor del hijo, luego rotar a la derecha alrededor del padre

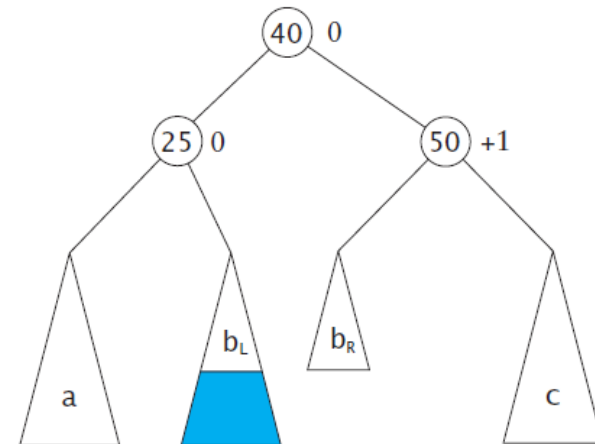
Insertion into b_L



Left Subtree After Rotate Left



Tree After Rotate Right

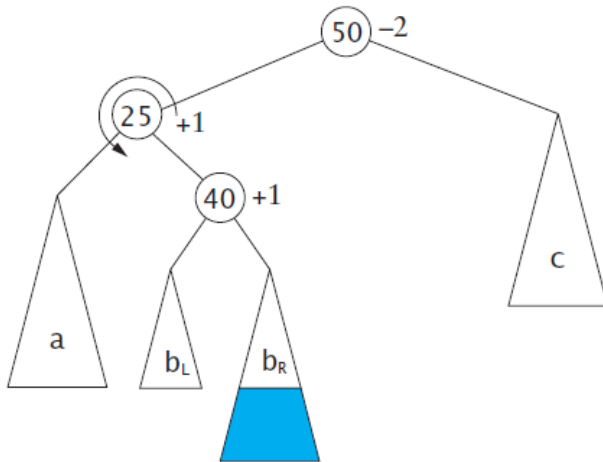


Árbol AVL (Adelson-Velskii y Landis)

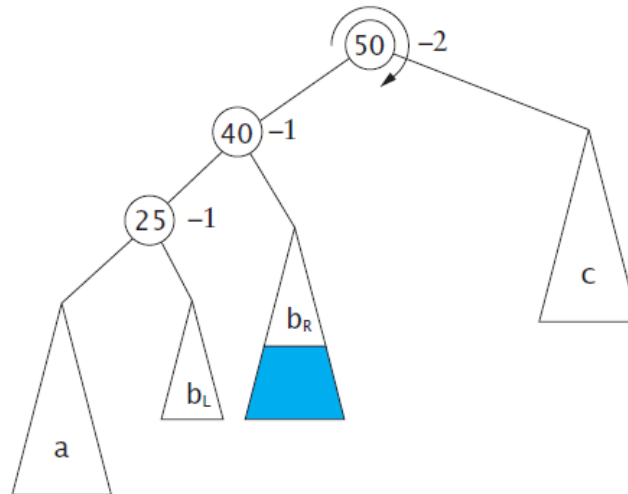
Tipos de rotaciones:

Izq-Der: padre tiene un desbalance de -2, hijo izq desbalance de +1 >> Rotar a la izq alrededor del hijo, luego rotar a la derecha alrededor del padre

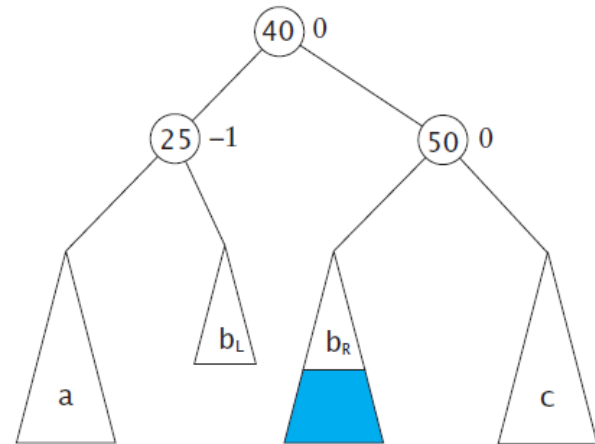
Insertion into b_R



Left Subtree After Rotate Left



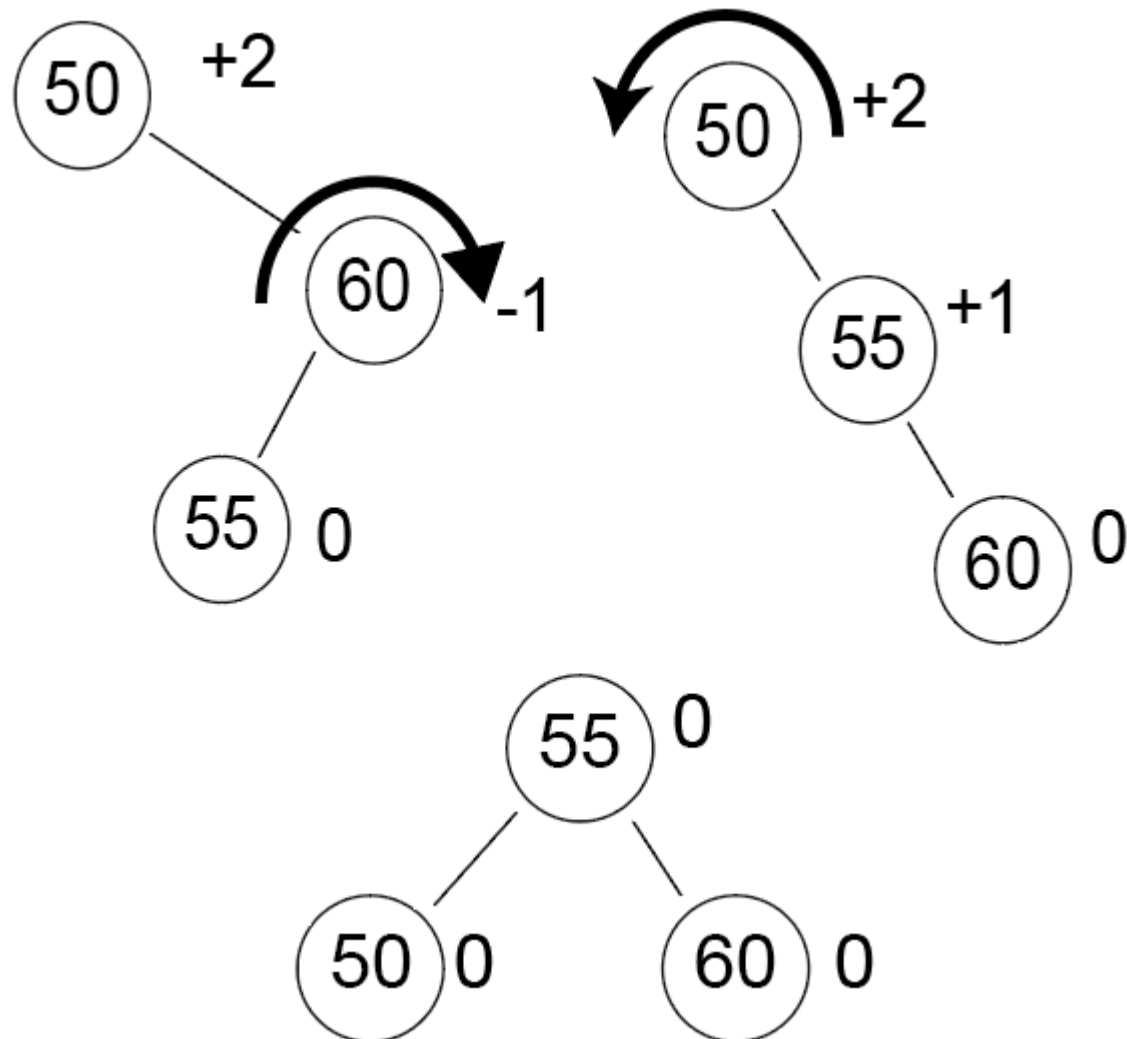
Tree After Rotate Right



Árbol AVL (Adelson-Velskii y Landis)

Tipos de rotaciones Der-Izq: padre tiene un desbalance de +2, hijo der desbalance de -1 >> Rotar a la derecha alrededor del hijo, luego rotar a la izquierda alrededor del padre

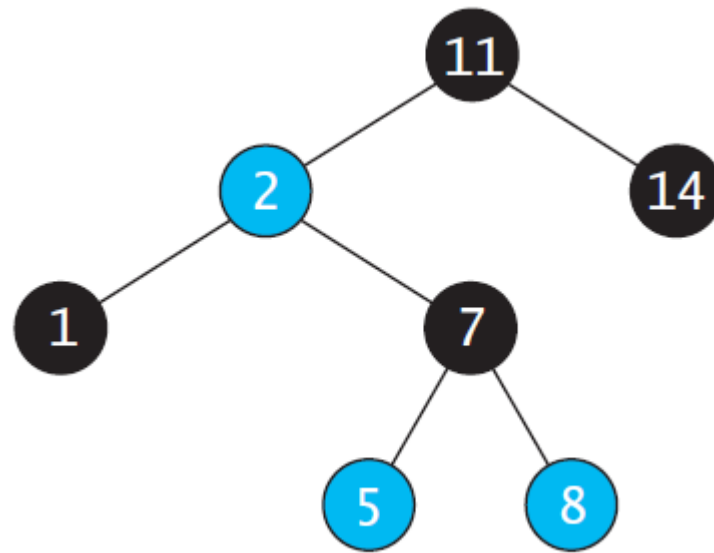
<https://www.cs.usfca.edu/~galles/visualization/AVLtree.html>



Árboles Rojo (azul) -Negro

1. Un nodo es rojo o negro.
2. La raíz siempre es negra.
3. Un nodo rojo siempre tiene hijos negros. (Se considera que un puntero nulo hace referencia a un nodo negro).
4. El número de nodos negros en cualquier ruta desde la raíz hasta una hoja es el mismo.

<https://www.cs.usfca.edu/~galles/visualization/RedBlack.html>



Árboles Splay

Un Splay Tree es un árbol binario de búsqueda que **no está balanceado explícitamente**, pero que utiliza una heurística de **auto-organización** para mantener los elementos más frecuentemente accedidos cerca de la raíz, optimizando el acceso a datos con **localidad temporal**.

Localidad Temporal: Propiedad de los patrones de acceso a datos donde existe una alta probabilidad de que un elemento recientemente utilizado sea utilizado nuevamente en un período de tiempo corto

<https://www.cs.usfca.edu/~galles/visualization/SplayTree.html>

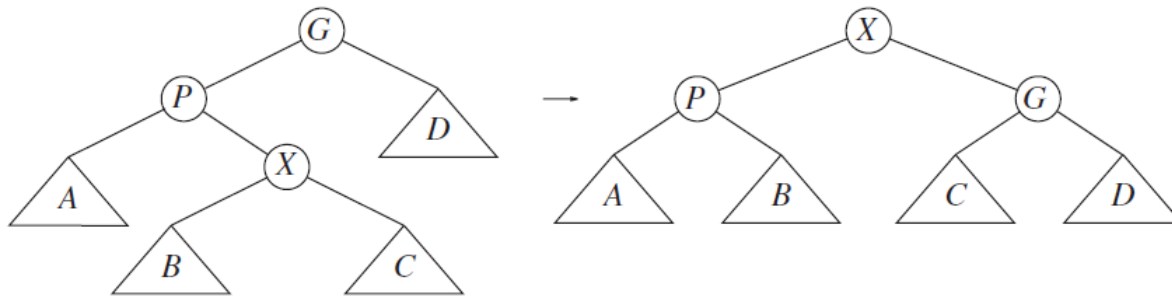


Figure 4.48 Zig-zag

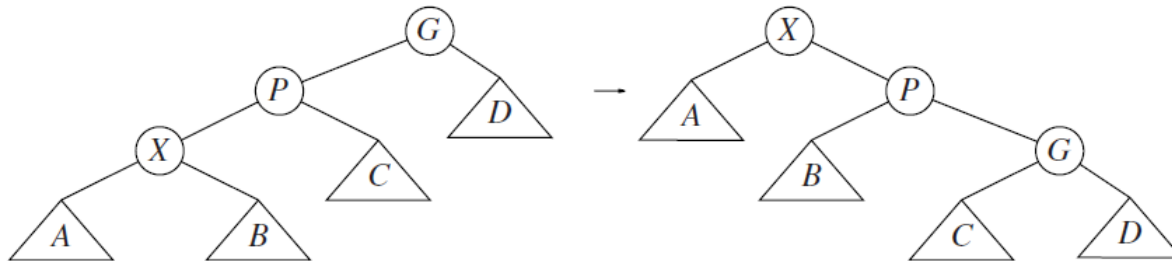


Figure 4.49 Zig-zig

ABB's

Característica	Splay Tree	AVL Tree	Red-Black Tree
Balance	No explícito	Estricto	Relajado
Altura garantizada	No	Sí ($\log n$)	Sí ($\sim \log n$)
Complejidad amortizada	$O(\log n)$	$O(\log n)$	$O(\log n)$
Peor caso	$O(n)$	$O(\log n)$	$O(\log n)$
Espacio extra	No	Sí (factor)	Sí (color)
Localidad temporal	Excelente	Normal	Normal
Implementación	Simple	Compleja	Muy compleja

Heaps y Colas de Prioridad

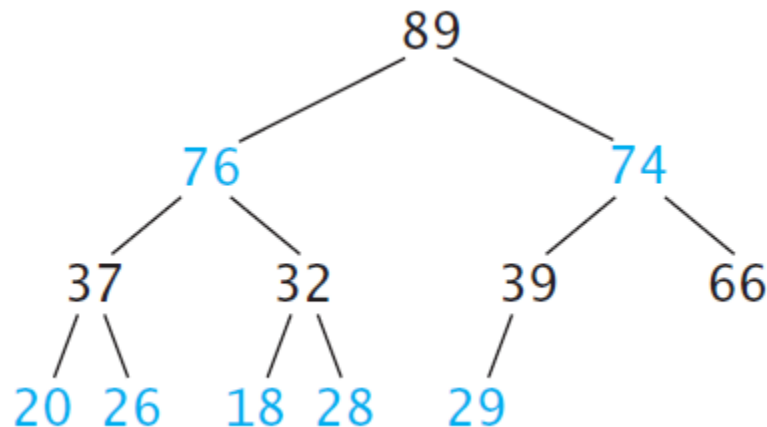
La estructura de datos Heap se define por dos propiedades:

Primero, es un árbol binario completo, por lo que los Heap casi siempre se implementan utilizando la representación de arreglo.

Segundo, los valores almacenados en un Heap están parcialmente ordenados. Esto significa que existe una relación entre el valor almacenado en cualquier nodo y los valores de sus hijos. Existen dos variantes del Heap, según la definición de esta relación:

Heap máximo tiene la propiedad de que cada nodo almacena un valor mayor o igual que el valor de cualquiera de sus hijos. Dado que la raíz tiene un valor mayor o igual que sus hijos, que a su vez tienen valores mayores o iguales que sus hijos, la raíz almacena el máximo de todos los valores del árbol.

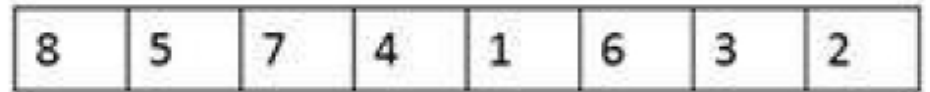
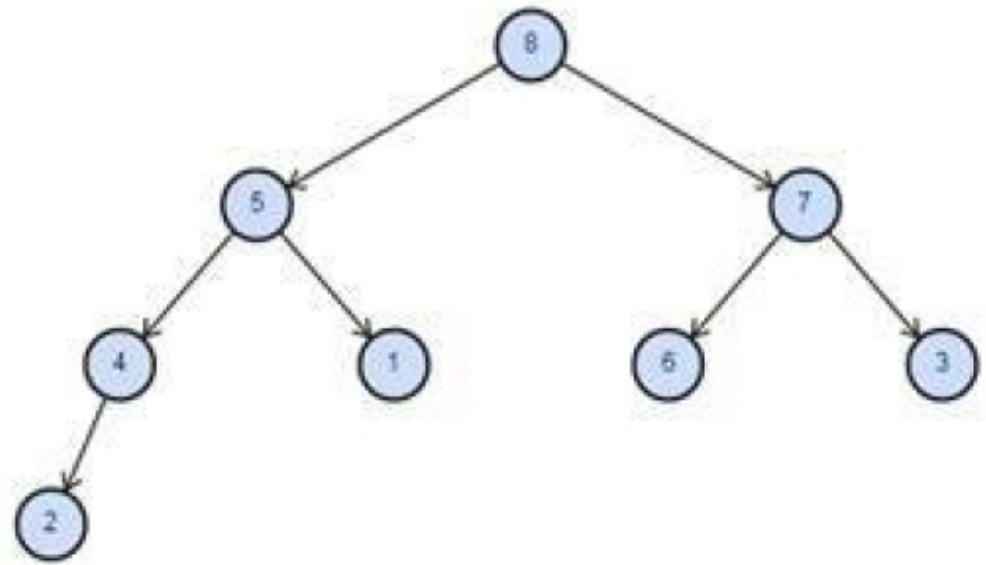
Heap mínimo tiene la propiedad de que cada nodo almacena un valor menor o igual que el de sus hijos. Dado que la raíz tiene un valor menor o igual que sus hijos, que a su vez tienen valores menores o iguales que sus hijos, la raíz almacena el mínimo de todos los valores en el árbol.



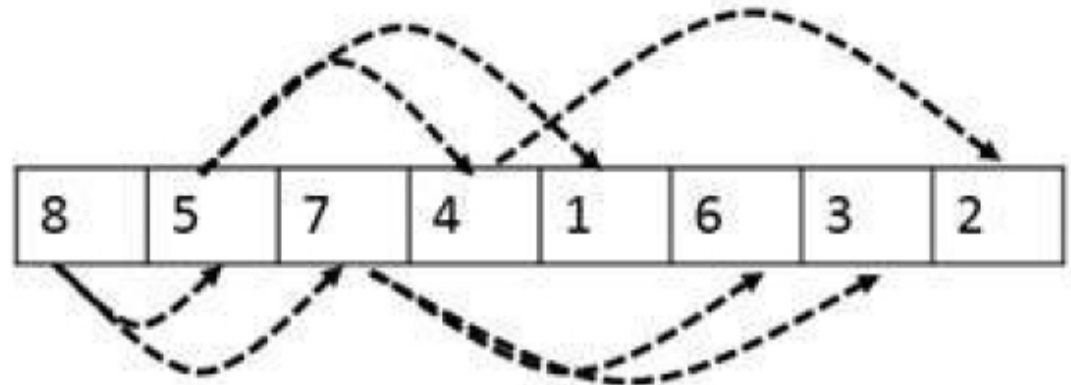
Heaps y Colas de Prioridad

A veces, una cola FIFO puede no ser la mejor manera de implementar una línea de espera. Existen muchas situaciones, tanto en la vida real como en aplicaciones informáticas, en las que deseamos elegir al siguiente "más importante" de un conjunto de personas, tareas u objetos. La prioridad se indica mediante un valor específico asociado al trabajo (y puede cambiar mientras este permanece en la lista de espera). **Cuando un conjunto de objetos se organiza por importancia o prioridad, lo llamamos**

cola de prioridad.



Max Heap



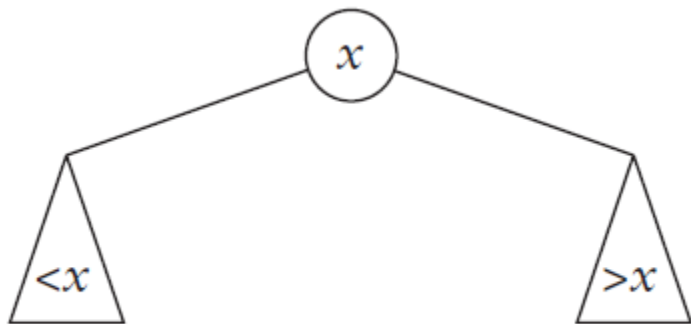
Árboles NO Binarios

Árbol 2-3

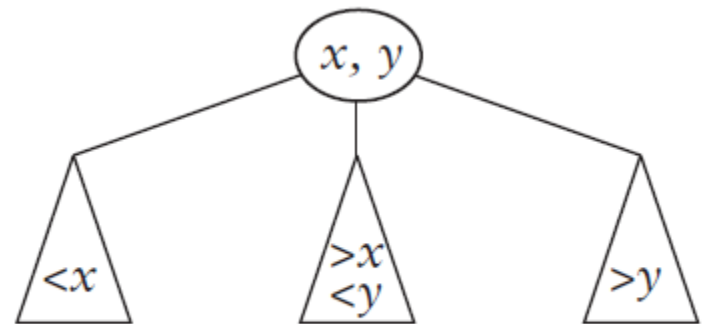
Un árbol 2-3 se compone de nodos denominados de 2 y 3 nodos.

Un nodo de 2 es similar a un nodo de árbol de búsqueda binario: consta de **un** campo de datos y referencias a dos hijos, uno con valores menores que el campo de datos y el otro con valores mayores que el campo de datos.

Un nodo de 3 contiene **dos** campos de datos, ordenados de modo que el primero sea menor que el segundo, y referencias a tres hijos: un hijo con valores menores que el primer campo de datos, un hijo con valores entre ambos campos de datos y un hijo con valores mayores que el segundo campo de datos.



2-node



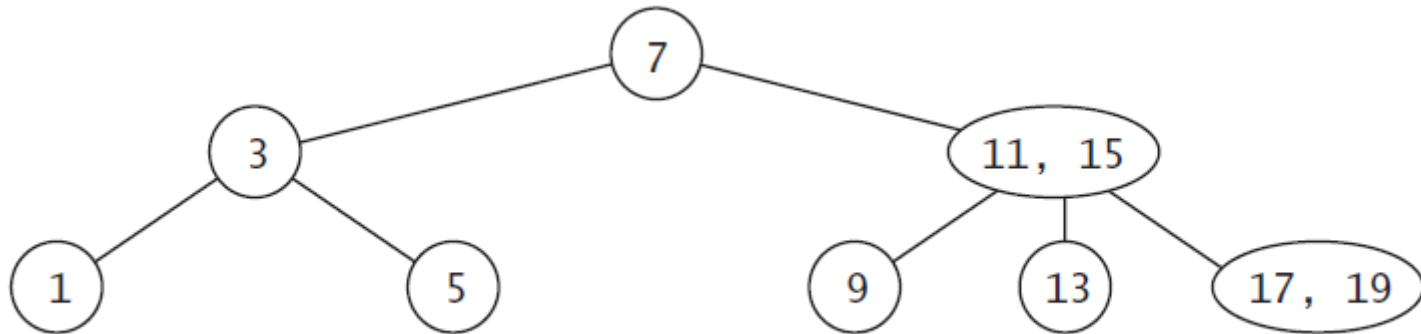
3-node

Árbol 2-3

Un árbol 2-3 se compone de nodos denominados de 2 y 3 nodos.

Un nodo de 2 es similar a un nodo de árbol de búsqueda binario: consta de un campo de datos y referencias a dos hijos, uno con valores menores que el campo de datos y el otro con valores mayores que el campo de datos.

Un nodo de 3 contiene dos campos de datos, ordenados de modo que el primero sea menor que el segundo, y referencias a tres hijos: un hijo con valores menores que el primer campo de datos, un hijo con valores entre ambos campos de datos y un hijo con valores mayores que el segundo campo de datos.



Árbol 2-3

FIGURE 11.35

Inserting into a Tree with All 2-Nodes

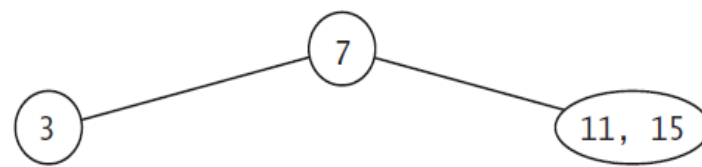
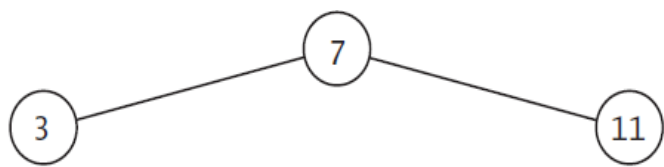


FIGURE 11.36

A Virtual Insertion

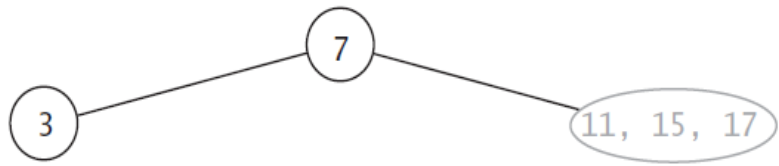


FIGURE 11.37

Result of Propagating 15 to 2-Node Parent

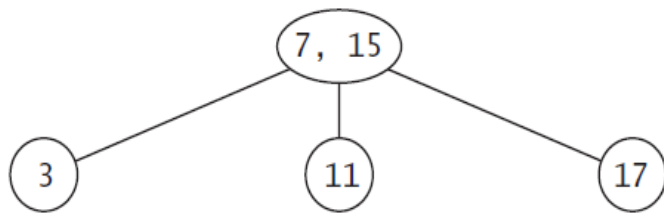
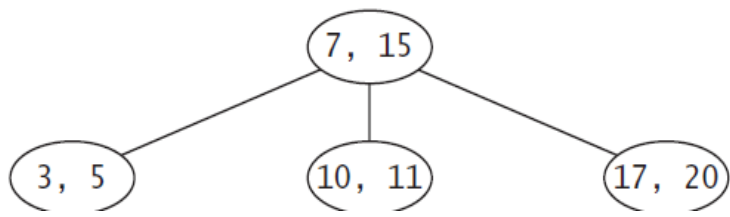


FIGURE 11.38

Inserting 5, 10, and 20



Árbol 2-3

FIGURE 11.39

Virtually Inserting 13

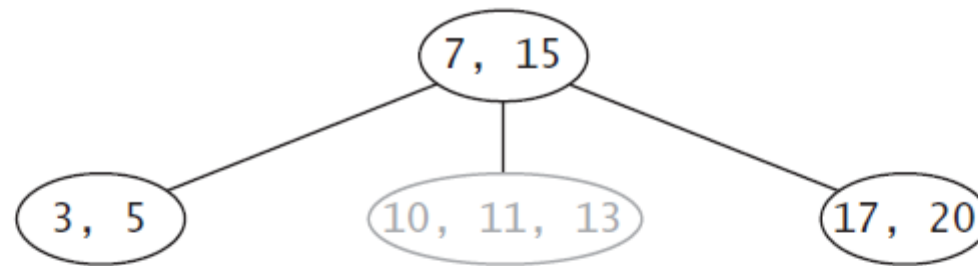


FIGURE 11.40

Virtually Inserting 11

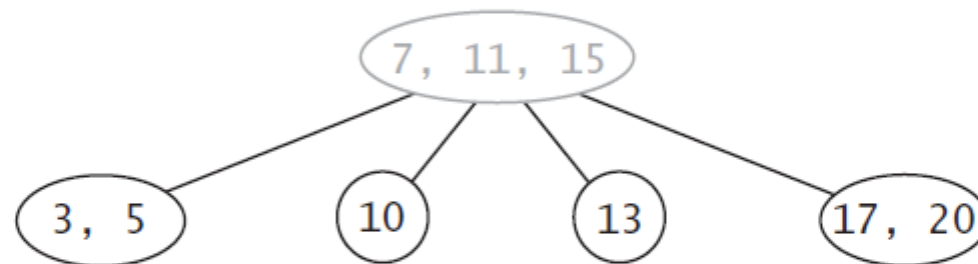
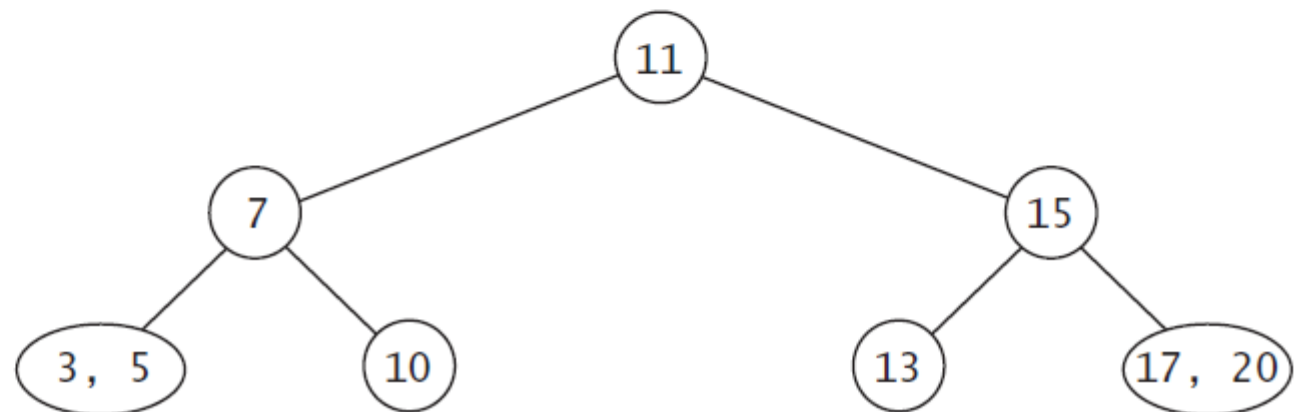


FIGURE 11.41

Result of Making 11 the New Root



Árbol 2-3

Análisis de árboles 2-3 y comparación con árboles binarios balanceados

El árbol 2-3 resultante del ejemplo anterior es un árbol balanceado de **altura 3** que requiere menos manipulaciones complejas. No se realizaron rotaciones, como se necesitó para construir los árboles AVL y Rojo-Negro, ambos de **altura 4**.

El número de elementos que puede contener un árbol 2-3 de altura h está entre $2^h - 1$ (todos de 2 nodos) y $3^h - 1$ (todos de 3 nodos). Por lo tanto, la altura de un árbol 2-3 está entre $\log_3 n$ y $\log_2 n$.

El tiempo de búsqueda es $O(\log n)$, ya que todos los logaritmos están relacionados por un factor constante, y los factores constantes se ignoran en la notación big-O.

Para **$n = 100$ elementos**:

Altura mínima:

$$\begin{aligned} h_{\text{mín}} &= \lceil \log_3 100 \rceil \\ &= \lceil 4.19 \rceil \\ &= 5 \text{ niveles} \end{aligned}$$

Altura máxima:

$$\begin{aligned} h_{\text{máx}} &= \lceil \log_2 100 \rceil \\ &= \lceil 6.64 \rceil \\ &= 7 \text{ niveles} \end{aligned}$$

Rango de altura: $5 \leq h \leq 7$

Árbol 2-3

Representación dinámica

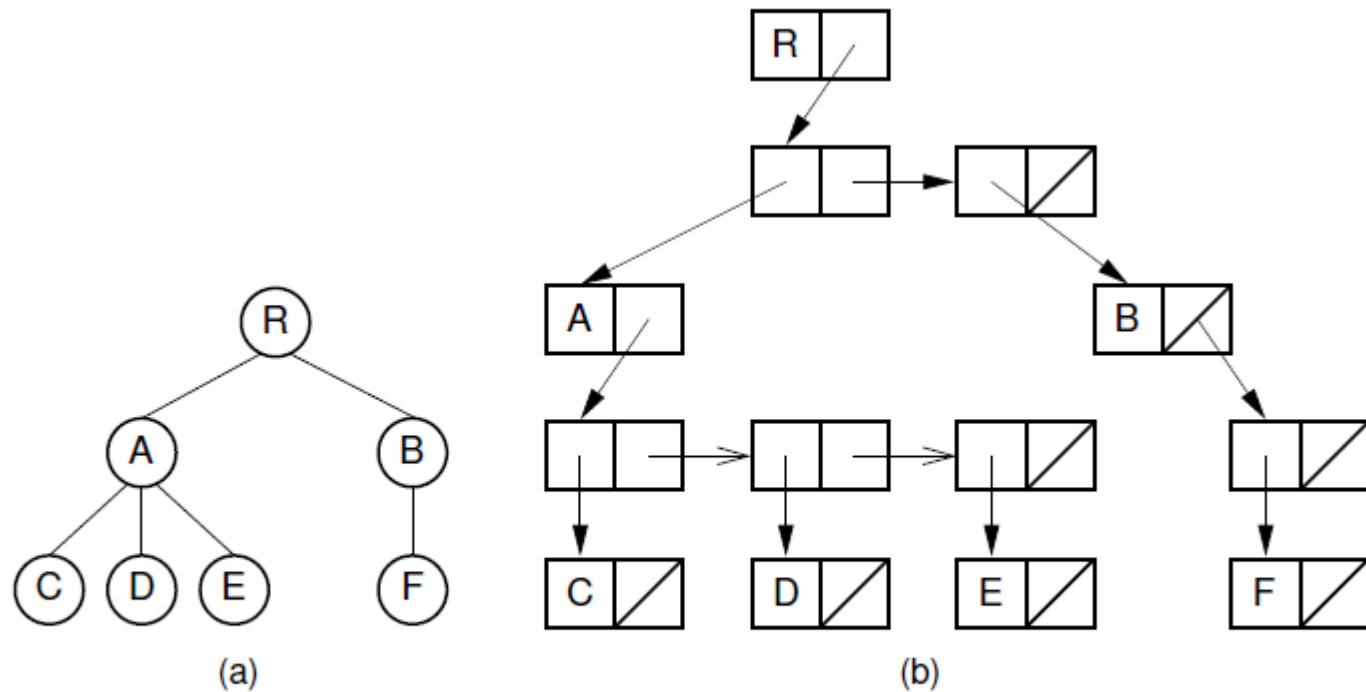
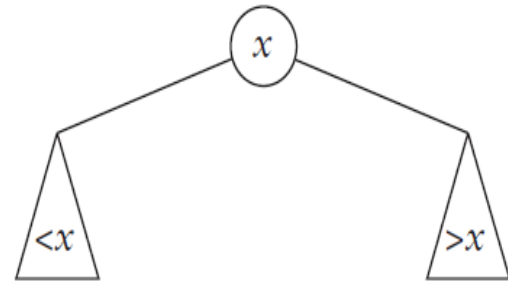


Figure 6.13 A dynamic general tree representation with linked lists of child pointers. (a) The general tree. (b) The tree representation.

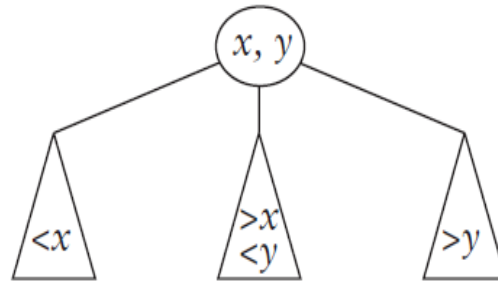
Árbol 2-3-4

Los árboles 2-3-4 amplían la idea de los árboles 2-3 añadiendo el nodo 4. Este es un nodo con tres elementos de datos y cuatro hijos.

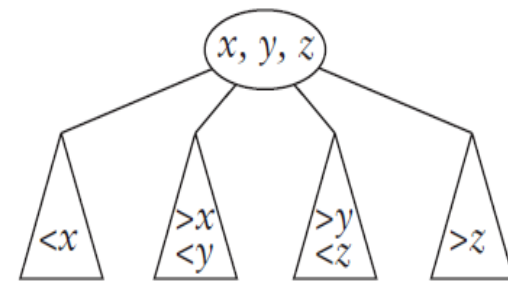
2-, 3-, and 4-Nodes



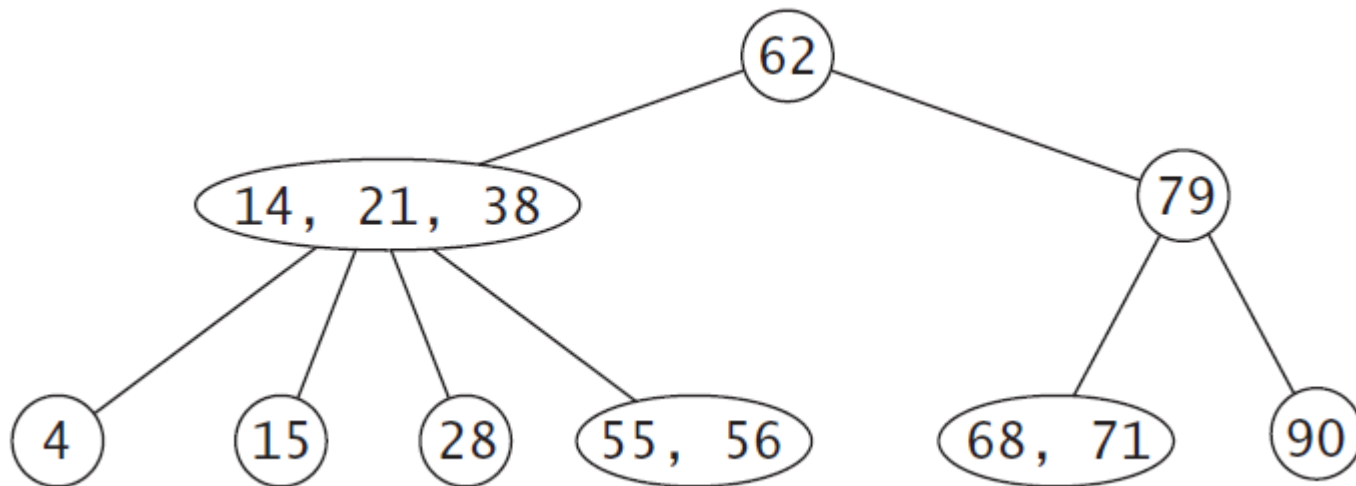
2-node



3-node



4-node



Árbol B

Un Árbol B es una estructura de datos auto-balanceada en forma de árbol que está optimizada para sistemas que leen y escriben grandes bloques de datos, como los sistemas de archivos y las bases de datos. A diferencia de los árboles binarios, los nodos de un árbol B pueden tener múltiples hijos, desde un límite inferior hasta un límite superior.

Aquí están las propiedades clave que definen un árbol B de orden m (donde $m \geq 2$ es un entero):

1. Número de Hijos:

Cada nodo interno (no hoja) tiene como mínimo $\lceil m/2 \rceil$ hijos y como máximo m hijos. La raíz puede tener entre 2 y m hijos (a menos que sea una hoja).

2. Número de Claves:

Un nodo interno con k hijos contiene $k-1$ claves.

Por lo tanto, cada nodo interno (que no es la raíz) tiene al menos $\lceil m/2 \rceil - 1$ claves y como máximo $m-1$ claves.

3. Ordenamiento de las Claves:

Las claves dentro de cada nodo se mantienen en orden creciente.

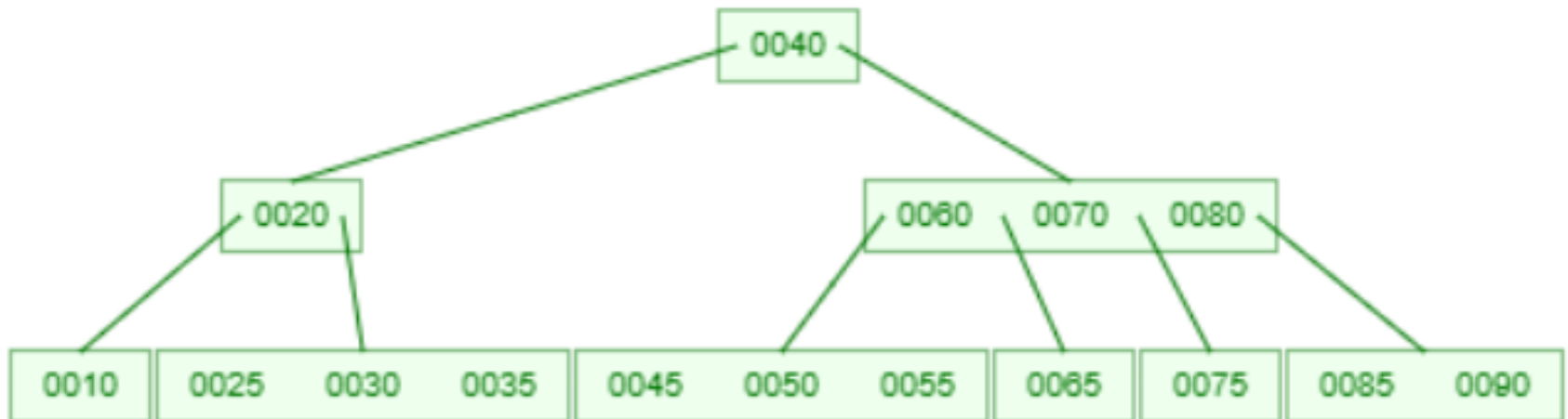
4. Relación entre Claves e Hijos:

Si un nodo tiene k hijos, entonces tiene $k-1$ claves ($c_1, c_2, \dots, c_{k-1}$). Estas claves dividen los rangos de las claves almacenadas en cada subárbol hijo

Árbol B

Un Árbol B es una estructura de datos auto-balanceada en forma de árbol que está optimizada para sistemas que leen y escriben grandes bloques de datos, como los sistemas de archivos y las bases de datos. A diferencia de los árboles binarios, los nodos de un árbol B pueden tener múltiples hijos, desde un límite inferior hasta un límite superior.

- ☒ Max. Degree = 4
- ☐ Max. Degree = 5
- ☐ Max. Degree = 6
- ☐ Max. Degree = 7



Árbol B+

Una variación de los árboles B donde todos los datos se almacenan en los nodos hoja, y los nodos internos solo contienen claves para guiar la búsqueda. Los nodos hoja a menudo están enlazados entre sí, lo que facilita las búsquedas de rango eficientes.

1. Almacenamiento de Datos:

Árbol B: Puede almacenar tanto claves como datos asociados en sus nodos internos y en las hojas. Esto significa que la información que buscas podría encontrarse en cualquier nivel del árbol.

Árbol B+: Solo almacena las claves en los nodos internos, que actúan como índices para dirigir la búsqueda. Los datos asociados a las claves se almacenan únicamente en las hojas. Las hojas contienen todas las claves y los punteros a los datos reales.

2. Estructura de las Hojas:

Árbol B: Las hojas son como cualquier otro nodo, simplemente en el nivel más bajo, y pueden contener claves y datos. No tienen una estructura especial que las conecte.

Árbol B+: Las hojas están enlazadas entre sí, formando una lista enlazada secuencial. Esto permite un recorrido eficiente de todos los datos en orden secuencial de las claves.

3. Búsqueda:

Árbol B: La búsqueda puede terminar en cualquier nodo si se encuentra la clave y

Árbol B+

4. Inserción y Eliminación (I/E):

Árbol B: La I/E pueden ser más complejas, ya que los datos pueden estar en nodos internos, lo que podría requerir ajustes más extensos en la estructura árbol.

Árbol B+: Estas operaciones tienden a ser más sencillas porque solo afectan directamente a las hojas (donde se almacenan los datos). Las modificaciones en los nodos internos se centran principalmente en mantener la estructura del índice.

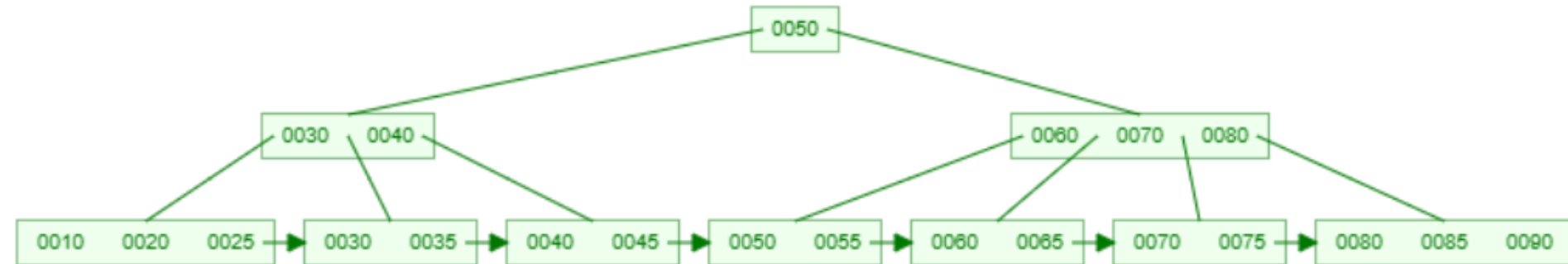
5. Consultas de Rango:

Árbol B: Las consultas de rango (buscar todas las claves dentro de un cierto intervalo) pueden requerir subir y bajar por diferentes partes del árbol.

Árbol B+: Son mucho más eficientes para las consultas de rango debido a la lista enlazada de las hojas. Una vez que se encuentra la primera clave en el rango, se puede recorrer secuencialmente la lista enlazada de las hojas para encontrar todas las demás claves dentro del rango.

Árbol B+

- ☒ Max. Degree = 4
- ☐ Max. Degree = 5
- ☐ Max. Degree = 6
- ☐ Max. Degree = 7



En resumen:

Característica	Árbol B	Árbol B+
Datos	En nodos internos y hojas	Solo en las hojas; nodos internos son índices
Hojas	No enlazadas	Enlazadas como una lista secuencial
Búsqueda	Puede terminar en cualquier nodo	Siempre llega a una hoja
Rango de Consultas	Menos eficiente	Más eficiente debido a la lista enlazada de hojas
Complejidad de I/E	Generalmente más complejo	Generalmente más simple, enfocado en las hojas

Visualizaciones / simulaciones de algoritmos

<https://www.cs.usfca.edu/~galles/visualization/Algorithms.html>

AVL <https://www.cs.usfca.edu/~galles/visualization/AVLtree.html>

B-Tree <https://www.cs.usfca.edu/~galles/visualization/BTree.html>

B+ -Tree <https://www.cs.usfca.edu/~galles/visualization/BPlusTree.html>

UNC
Fac. Ciencias Exactas
Físicas y Naturales

AED – Búsqueda

BÚSQUEDA

La búsqueda es el proceso de localizar un elemento específico dentro de una estructura de datos.

Se aplica en listas, arreglos, bases de datos, estructuras jerárquicas.

Elegir el algoritmo adecuado impacta en la eficiencia del sistema.

BÚSQUEDA

Consideraciones

Complejidad Temporal:

- **Mejor caso** (ej: búsqueda lineal el dato está en el primer lugar)
- **Caso promedio**, comportamiento esperado con entradas aleatorias
- **Peor caso**, escenario menos favorable posible

Complejidad Espacial:

Memoria adicional requerida, crítico para sistemas con recursos limitados o grandes volúmenes de datos (mem principal o auxiliar, iterativo o recursivo)

Frecuencia de operaciones:

Búsquedas frecuentes vs pocas: Justifica estructuras más complejas

Proporción búsqueda/inserción/eliminación: Influye en la elección óptima

BÚSQUEDA

Restricciones:

- ¿Los datos están ordenados?

Ordenados: Algunos algoritmos (binario, interpolación) requieren ordenamiento previo

No ordenados: Limita opciones pero evita preprocesamiento

Preprocesamiento: ¿El costo de ordenar/preparar compensa el beneficio en búsquedas?

- ¿Estructura de datos subyacente?

Arrays: Acceso directo por índice, memoria contigua

Listas enlazadas: Solo acceso secuencial, memoria dispersa

Árboles/Grafos: Relaciones jerárquicas o en red

Tablas hash: Optimizadas para búsqueda por clave

- ¿Acceso aleatorio vs secuencial?

Acceso aleatorio: Capacidad de saltar directamente a cualquier posición ($O(1)$)

Acceso secuencial: Solo se puede acceder en orden ($O(n)$ para alcanzar posición n)

Búsqueda Lineal (Secuencial)

Idea Principal:

Examinar cada elemento de la colección hasta encontrar (o no) el objetivo

Características:

Simple de implementar

Funciona en colecciones no ordenadas

No requiere estructura de datos especial

Ineficiente en grandes volúmenes de datos

Complejidad:

Tiempo: $O(n)$

Espacio: $O(1)$

85	21	62	51	94	8	7	81	6	19
0	1	2	3	4	5	6	7	8	9

Pseudocódigo

función búsquedaLineal(arreglo, objetivo):

 para i desde 0 hasta longitud(arreglo)-1:

 si arreglo[i] == objetivo:

 retornar i

 retornar -1 // No encontrado

Búsqueda Binaria

Idea Principal:

Dividir repetidamente por la mitad el espacio de búsqueda

Prerequisito:

Datos deben estar ordenados

Características:

Muy eficiente para colecciones grandes

Usa el paradigma de "divide y vencerás"

Requiere acceso aleatorio a los elementos

Complejidad:

Tiempo: $O(\log n)$

Espacio: $O(1)$ [iterativa] / $O(\log n)$ [recursiva]

Búsqueda Binaria

Pseudocodigo:

inicio = 0, fin = n – 1

mientras inicio <= fin:

 medio = (inicio + fin) / 2

 si A[medio] == clave:

 retornar medio

 si A[medio] < clave:

 inicio = medio + 1

 sino: fin = medio – 1

retornar -1

85	106	168	219	313	321	328	409	415	434
0	1	2	3	4	5	6	7	8	9

Búsqueda en ABB

Idea principal:

Cada nodo tiene mayor valor que su subárbol izquierdo

Cada nodo tiene menor valor que su subárbol derecho

AVL: Factor de balance máximo de ± 1

Rojo-Negro

B-Trees: Optimizados para almacenamiento externo

Características:

Eficiente para conjuntos dinámicos de datos.

Permite búsquedas, inserciones y eliminaciones en tiempo logarítmico si está balanceado

Puede degradarse a una lista enlazada si no se balancea, en ese caso, las operaciones se vuelven $O(n)$

Inserciones/eliminaciones más eficientes vs Array: $O(\log n)$ vs $O(n)$

Complejidad:

Mejor caso $O(\log n)$

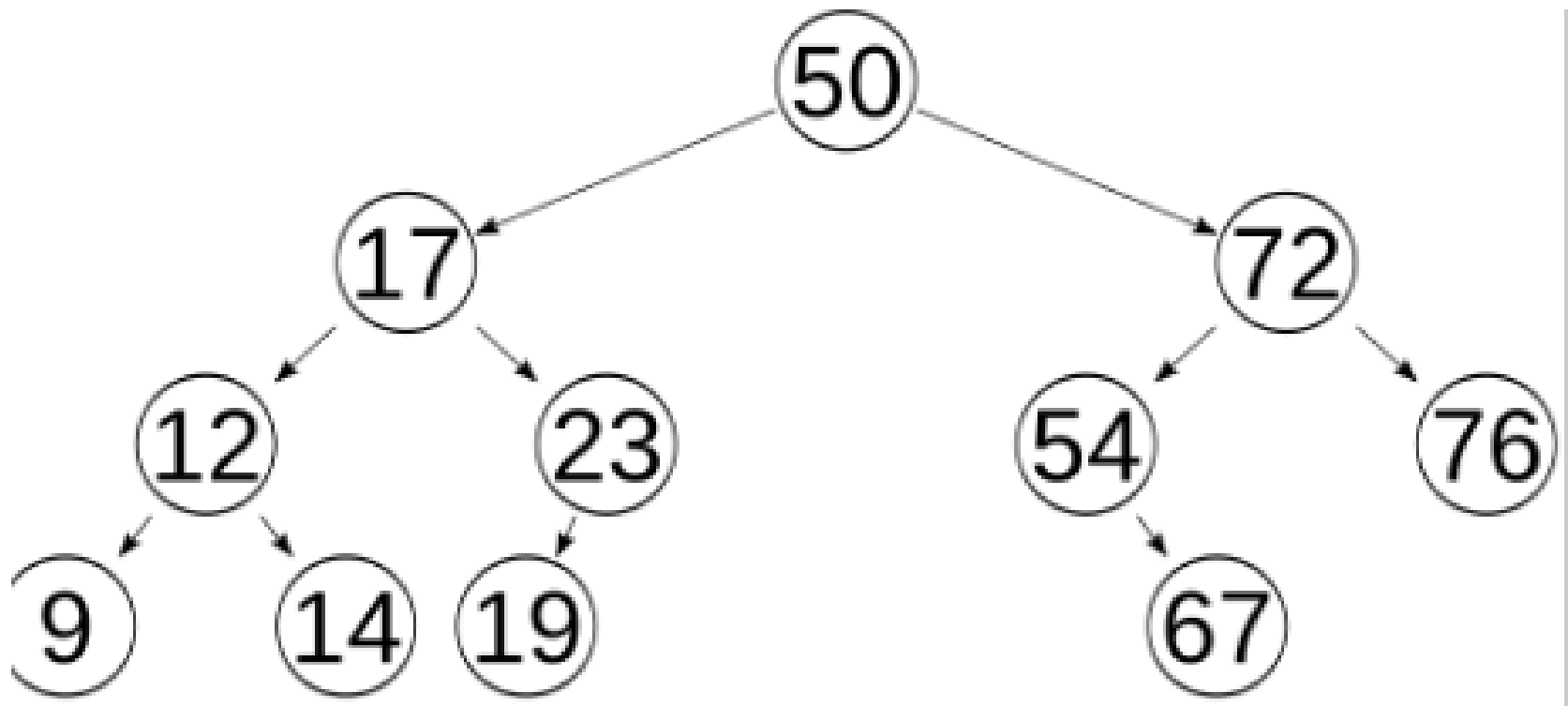
Promedio $O(\log n)$

Peor caso $O(n)$ (árbol degenerado)

Búsqueda en ABB

Cada nodo tiene mayor valor que su subárbol izquierdo

Cada nodo tiene menor valor que su subárbol derecho



Búsqueda por Interpolación

Idea principal:

Similar a la búsqueda binaria, pero en lugar de siempre dividir por la mitad, estima la posición del objetivo basándose en su valor relativo dentro del rango actual.

Prerequisito:

Datos deben estar ordenados

Aproximadamente uniformemente distribuidos

Características:

Puede ser más rápida que la binaria si la distribución es uniforme

Calcula una posición aproximada del elemento

Fórmula:
$$\text{pos} = \text{izq} + ((\text{objetivo} - \text{arreglo}[\text{izq}]) * (\text{der} - \text{izq})) / (\text{arreglo}[\text{der}] - \text{arreglo}[\text{izq}])$$

Complejidad:

Promedio: $O(\log \log n)$ para datos uniformemente distribuidos.

Peor Caso: $O(n)$ (si los datos no están uniformemente distribuidos)

Búsqueda por Interpolación

Calcula una posición aproximada del elemento

Fórmula:
$$\text{pos} = \text{izq} + ((\text{objetivo} - \text{arreglo}[\text{izq}]) * (\text{der} - \text{izq})) / (\text{arreglo}[\text{der}] - \text{arreglo}[\text{izq}])$$

Arreglo ordenado uniforme: [10, 20, 30, 40, 50, 60, 70, 80, 90]

Búsqueda del valor 70:

$$\text{pos} = 0 + ((70-10)*(8-0))/(90-10)$$

$$\text{pos} = 0 + 480/80 = 0 + 6 = 6$$

arreglo[6] = 70 ✓ ¡Encontrado en la primera estimación!

Comparación con binaria:

Binaria: 2 comparaciones (4, 6)

Interpolación: 1 comparación (para distribución uniforme)

Búsqueda Probabilística (aleatoria)

Idea principal:

es un enfoque en el que las decisiones de búsqueda se hacen de manera aleatoria o basada en heurísticas (soluciones a problemas de manera rápida y eficiente, especialmente cuando no hay una solución clara o cuando el tiempo es limitado). Es útil cuando no tenemos conocimiento claro sobre la distribución de los datos.

Prerequisito:

Los datos pueden estar desordenados

Características:

Cuando la estructura de los datos es desconocida o cambia frecuentemente.
En espacios de búsqueda muy grandes, como juegos, optimización, IA.

Ventajas: Simple en ciertos contextos.

Puede ser rápida en promedio. Escalable y adaptable.

Desventajas: No garantiza éxito en tiempo finito.

Difícil de analizar formalmente.

Puede necesitar múltiples iteraciones

Búsqueda Probabilística (aleatoria)

Complejidad:

Promedio: variable

Peor Caso: $O(n)$ o indefinido

Mejor Caso: $O(1)$

```
int busqueda_aleatoria(int A[], int n, int clave) {  
    vector<bool> visitado(n, false);  
    int intentos = 0;  
    while (intentos < n) {  
        int i = rand() % n;  
        if (!visitado[i]) {  
            visitado[i] = true;  
            if (A[i] == clave)  
                return i;  
            intentos++;  
        }  
    }  
    return -1; // No encontrado  
}
```

Tabla de Dispersión - Hash

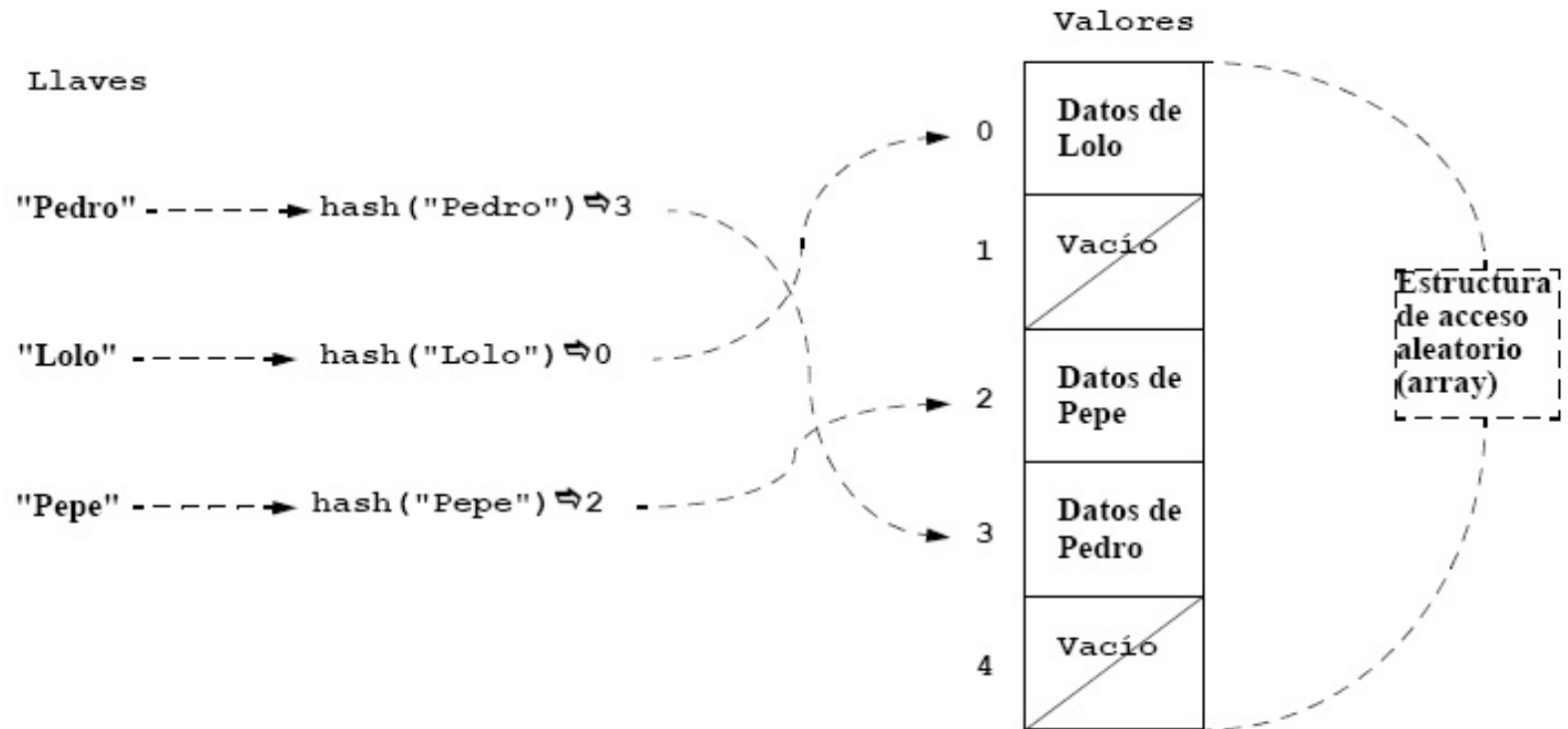
El TDA Tabla de Dispersión es un tipo de datos homogéneo de tamaño fijo, compuesto por un número fijo de componentes, denominadas celdas, a las que se accede mediante una dirección de memoria resultante de una función de transformación.

Sobre este TDA se definen los operadores Insertar, Buscar y Eliminar

La Dispersión es una técnica empleada para realizar inserciones, eliminaciones y búsquedas en un tiempo promedio constante $O(1)$.

Tabla de Dispersión

Esquema básico de una Tabla de Dispersión



hash () : función de dispersión (también llamada “función *hash*”)

Factor de carga $\lambda = \text{N}^\circ \text{ elementos en Tabla} / \text{tamaño de la Tabla}$

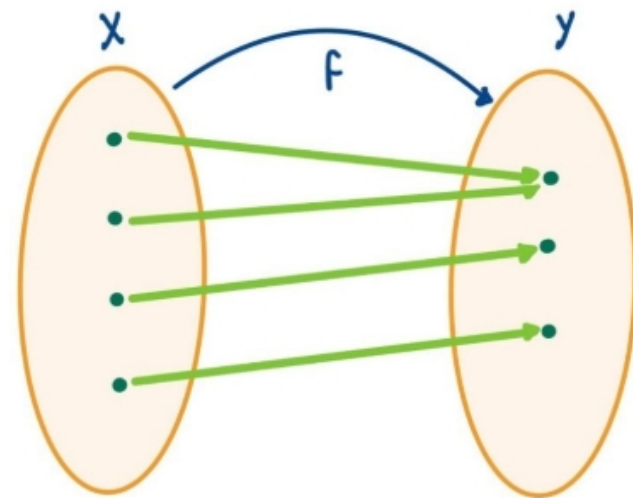
Tabla de Dispersión

Función de Dispersión

“Idealmente” debe generar una dirección de almacenamiento diferente para cada clave.

“Realmente” se busca que distribuya uniformemente las claves entre las direcciones posibles.

Definición: Dado un conjunto de claves posibles, X , y un conjunto de direcciones de memoria (Y), una función de transformación, $f(x)$, es una aplicación suprayectiva del conjunto de claves posibles en el conjunto de direcciones de memoria. $f: X \rightarrow Y$



Funciones de Dispersión

Ejemplos de Funciones de Dispersión

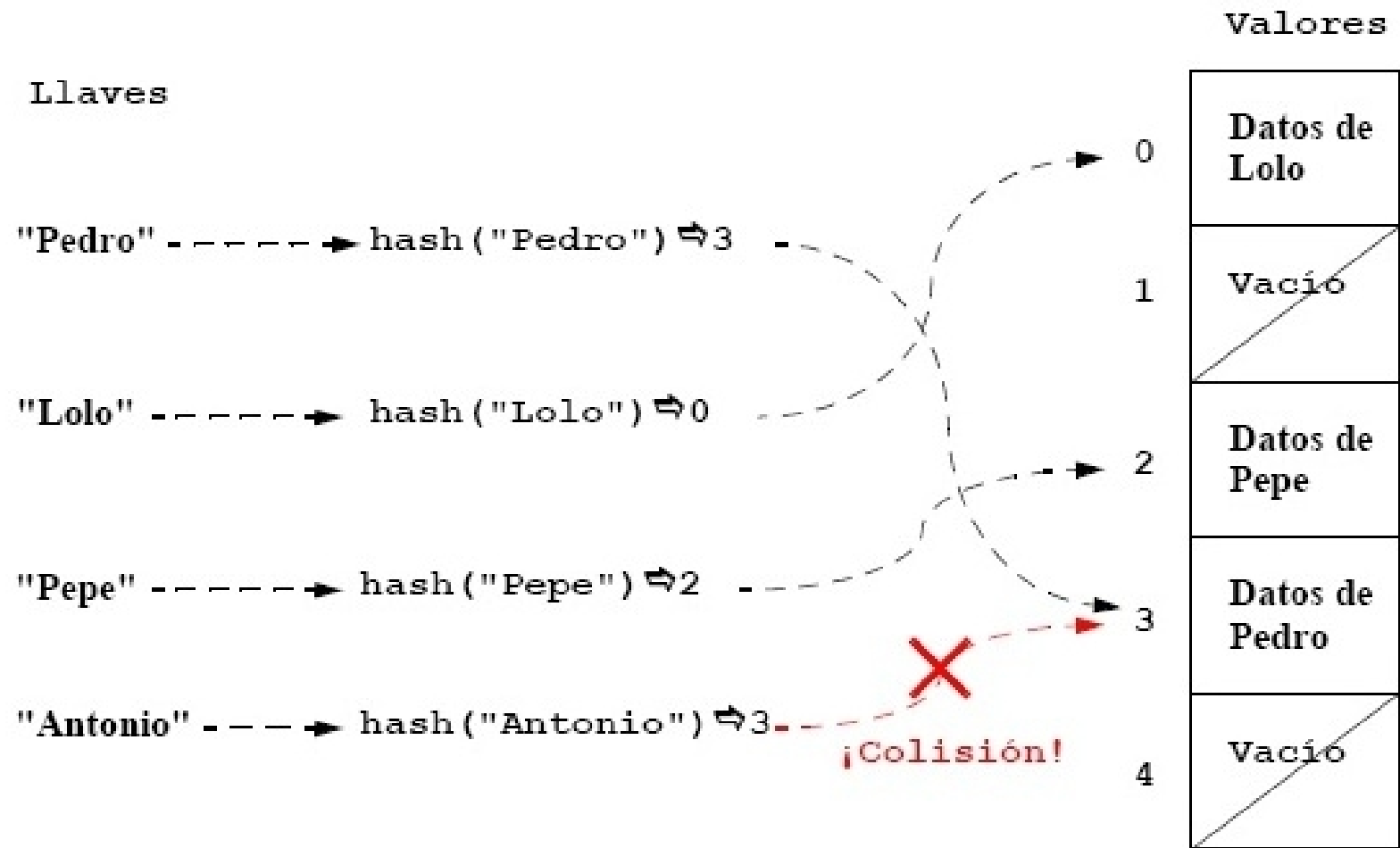
Para Claves Numéricas:

- Función módulo
- Troceado en grupos de cifras

Para Claves Alfanuméricas:

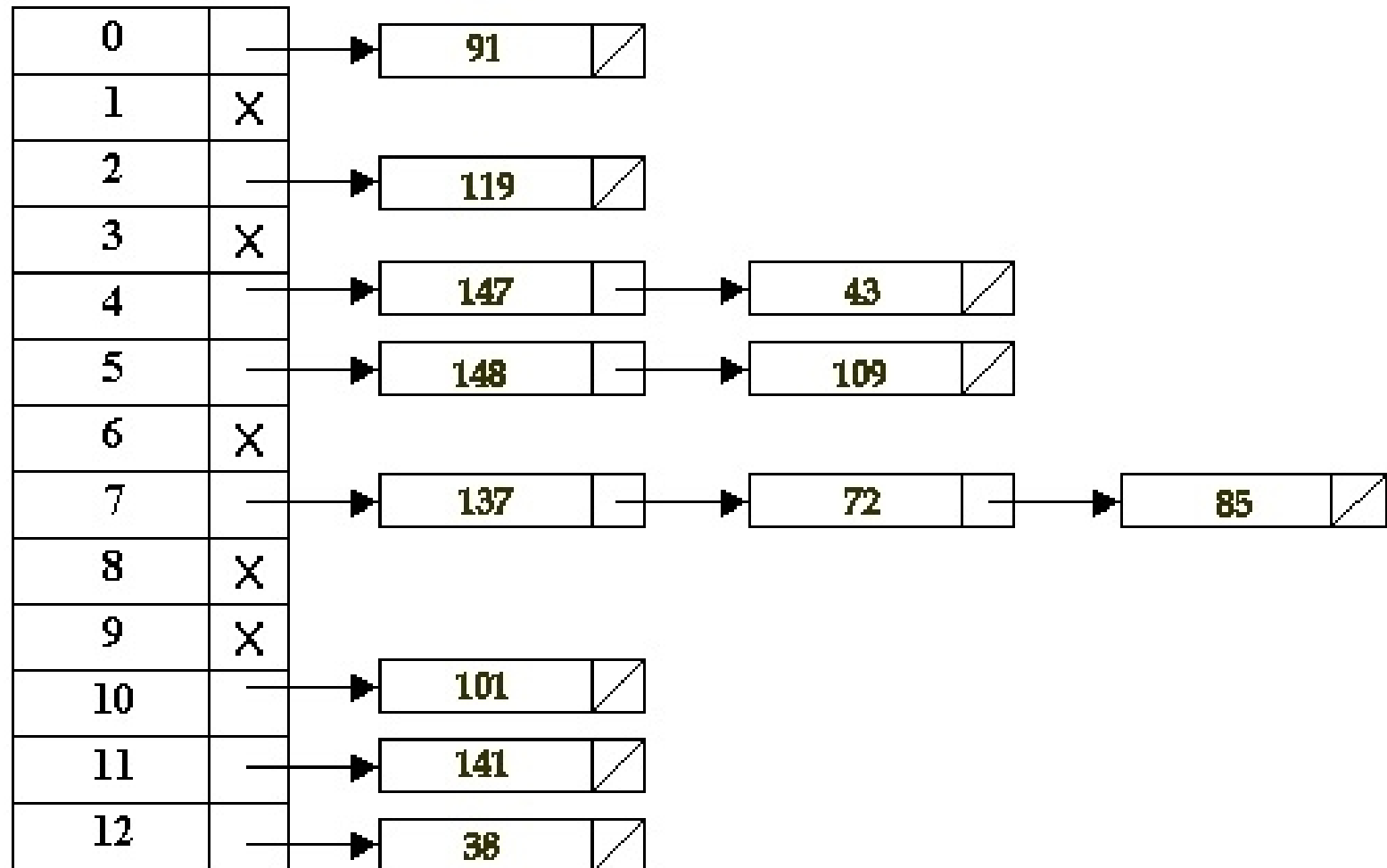
- Suma de los códigos ASCII de los caracteres
- Suma con pesos de los códigos ASCII de los caracteres

Colisiones



Manejo de Colisiones

Dispersión abierta



Manejo de Colisiones

Dispersión abierta, consiste en tener en una lista (u otra estructura de datos) a todos los elementos que se dispersan en la misma dirección.

Requiere en esencia la implantación de una segunda estructura de datos

Manejo de Colisiones

Dispersión cerrada, en caso de colisión se buscan direcciones alternativas hasta encontrar una vacía.

En la Dispersión cerrada, la búsqueda de direcciones alternativas puede ser por:

- Exploración lineal
- Exploración cuadrática
- Dispersión doble

Manejo de Colisiones

En la Dispersión cerrada, el factor de carga λ debe estar por debajo de 0,5. Gráfico del N° de intentos $\approx 1/(1-\lambda)$

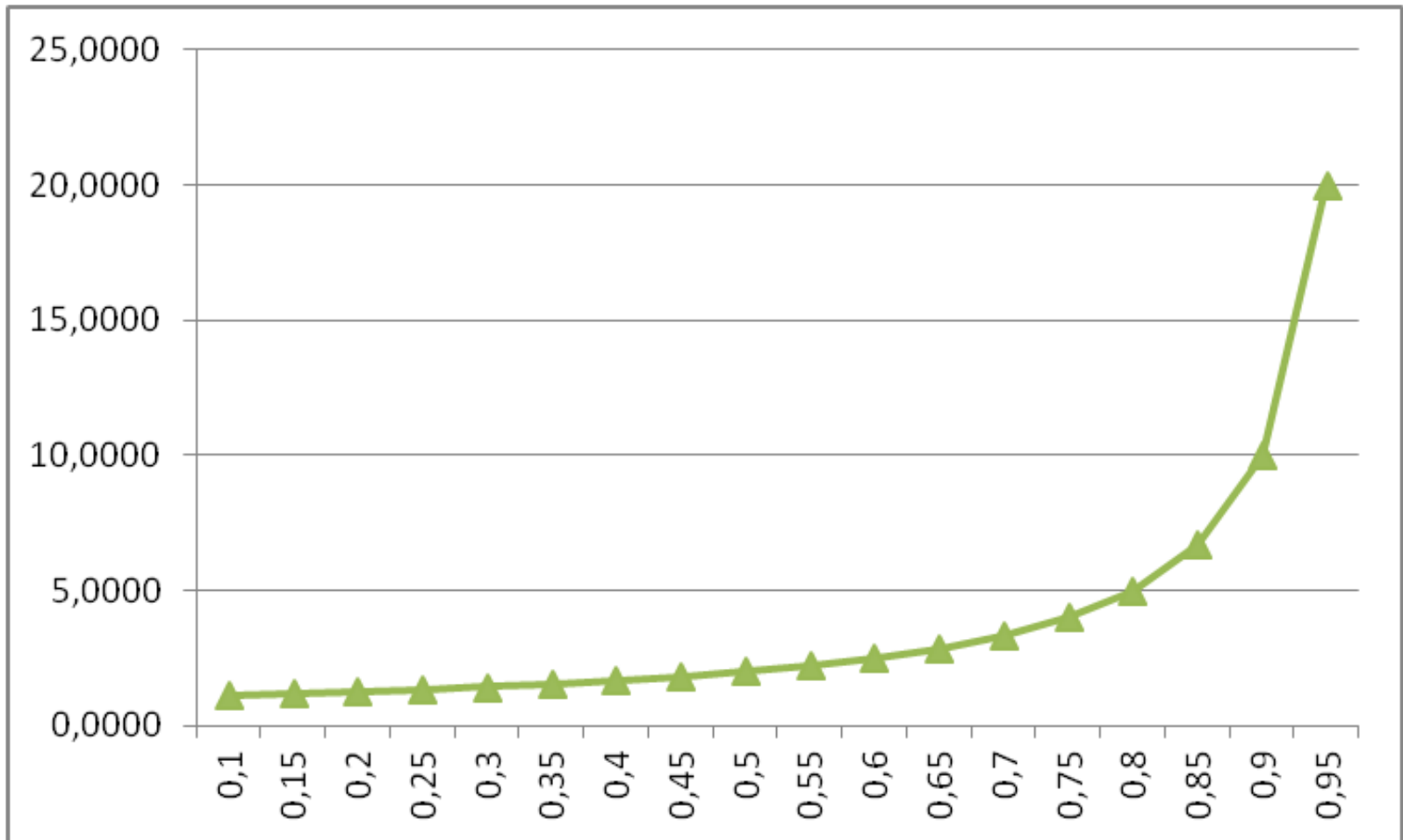


Tabla de dispersión

Resumen

Las Tablas de Dispersión se pueden usar para implantar las operaciones insertar y buscar en tiempo medio constante

Se debe conocer de antemano la cantidad de claves a almacenar para dimensionar la Tabla de Dispersión

Se complica cuando hay que borrar, sobre todo en dispersión cerrada

Es menos eficiente en el uso de memoria pues en general hay que prever el doble del número de claves

Comparativa de Algoritmos de Búsqueda

Comparativa de Algoritmos

Algoritmo	Tiempo (Mejor)	Tiempo (Promedio)	Tiempo (Peor)	Espacio	Prerequisitos
Lineal	$O(1)$	$O(n)$	$O(n)$	$O(1)$	Ninguno
Binaria	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$	Datos ordenados
Interpolación	$O(1)$	$O(\log \log n)$	$O(n)$	$O(1)$	Ordenado, distribución uniforme
Hash	$O(1)$	$O(1)$	$O(n)$	$O(n)$	Tabla hash
Árbol (BST)	$O(1)$	$O(\log n)$	$O(n)$	$O(n)$	Árbol de búsqueda
Árbol Balanceado	$O(1)$	$O(\log n)$	$O(\log n)$	$O(n)$	Árbol balanceado

Comparativa de Algoritmos de Búsqueda

Casos de Uso

Búsqueda Lineal:

- Colecciones pequeñas
- Datos no ordenados
- Búsquedas poco frecuentes

Búsqueda Binaria:

- Colecciones ordenadas grandes
- Cuando el costo de ordenamiento inicial se amortiza

Búsqueda Hash:

- Búsquedas muy frecuentes
- Sin necesidad de mantener orden

Búsqueda por Interpolación:

- Datos ordenados con distribución uniforme
- Arrays grandes con valores numéricos

Árboles de Búsqueda:

- Necesidad de operaciones ordenadas
- Colecciones dinámicas
- Búsquedas por rango

Búsqueda Probabilística:

- Grandes volúmenes de datos
- Cuando son aceptables respuestas aproximadas
- Espacio limitado (Bloom filters)

Comparativa de Algoritmos de Búsqueda

Conclusiones

No existe un algoritmo de búsqueda "mejor" para **todos** los casos

La elección depende de:

- Características de los datos
- Patrón de uso (frecuencia de búsqueda vs inserciones)
- Restricciones de memoria y tiempo
- Necesidad de operaciones adicionales

Algoritmos más eficientes generalmente requieren:

- Estructuras de datos más complejas
- Preprocesamiento
- Mayor uso de memoria

Conjunto (set)

Un conjunto es una colección desordenada de elementos únicos.

Operaciones típicas:

- Inserción
- Eliminación
- Búsqueda

Operaciones Fundamentales sobre Conjuntos:

Pertenencia: Verificar si un elemento está en el conjunto.

Ejemplo: ¿Está 'manzana' en {manzana, banana, naranja}?

Unión: Combinar dos conjuntos para obtener un nuevo conjunto con todos los elementos de ambos (sin duplicados). Notación: $A \cup B$.

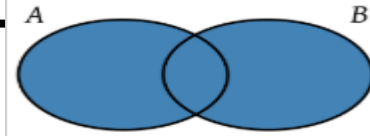
Intersección: Encontrar los elementos comunes a dos conjuntos. Notación: $A \cap B$.

Diferencia: Encontrar los elementos que están en un conjunto pero no en el otro. Notación: $A - B$.

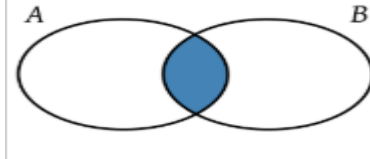
Subconjunto: Verificar si todos los elementos de un conjunto están contenidos en otro. Notación: $A \subseteq B$.

Cardinalidad: El número de elementos en un conjunto. Notación: $|A|$.

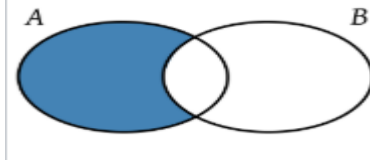
Operaciones con conjuntos



Unión



Intersección



Diferencia

Conjunto (set) - Implementación

Listas (o Arrays):

- Eficiencia de las operaciones fundamentales:
 - Pertenencia: $O(n)$ (necesita recorrer la lista).
 - Unión: Podría ser $O(n+m)$ si se evitan duplicados.
 - Intersección: Podría ser $O(n * m)$ en una implementación simple.
 - Inserción: $O(1)$ (al final) o $O(n)$ (si se mantiene orden para eficiencia en pertenencia).
 - Eliminación: $O(n)$ (necesita buscar el elemento)

Tablas Hash (Hash Sets):

- Eficiencia de las operaciones fundamentales (en promedio):
 - Pertenencia: $O(1)$.
 - Unión: $O(n+m)$.
 - Intersección: $O(\min(n,m))$.
 - Inserción: $O(1)$.
 - Eliminación: $O(1)$.

Conjunto (set) - Implementación

Árboles Binarios de Búsqueda (BST) Auto-Balanceados (e.g., AVL, Rojinegros):

Eficiencia de las operaciones fundamentales:

- Pertenencia: $O(\log n)$.
- Unión, Intersección, Diferencia: Pueden ser más complejas de implementar eficientemente pero generalmente en el orden de $O(n \log m)$ o similar dependiendo de la operación y tamaños.
- Inserción: $O(\log n)$.
- Eliminación: $O(\log n)$

Conjunto de bits (bitset)

Un conjunto es una colección desordenada de elementos únicos.

Operaciones típicas:

- Inserción
- Eliminación
- Búsqueda
- Unión / Intersección / Diferencia

Es una **estructura de datos compacta** que usa **bits** para representar la pertenencia de elementos a un conjunto.

Cada bit representa si un elemento está o no está en el conjunto

Ventajas

- Operaciones en tiempo constante para inserción, borrado, pertenencia.
- Eficiente para realizar operaciones de conjuntos (unión, intersección, etc.) mediante operaciones a nivel de bit.

Limitación

- Necesitas conocer el tamaño máximo del conjunto de antemano.
- Solo funciona con índices enteros (no strings ni objetos complejos).

Conjunto de bits (bitset)

Ventajas de usar Bitset para Conjuntos:

- Eficiencia en operaciones:** Muchas operaciones (pertenencia, inserción, eliminación, y las operaciones de conjuntos bit a bit) son muy rápidas.
- Uso eficiente de la memoria:** Si el universo no es demasiado grande, un bitset puede usar menos memoria que otras estructuras para representar conjuntos.

Desventajas:

- Limitación del universo:** Requiere un universo de elementos conocido y mapeable a índices dentro del tamaño del bitset. No es adecuado para conjuntos con elementos arbitrarios o universos muy grandes

Conjunto A = {1, 3, 5}

Conjunto B = {3, 6}

Bitset para A (tamaño 8): 0 1 0 1 0 1 0 0

Índices: 0 1 2 3 4 5 6 7

Bitset para B (tamaño 8): 0 0 0 1 0 0 1 0

Índices: 0 1 2 3 4 5 6 7

Unión (A U B): OR bit a bit -> 0 1 0 1 0 1 1 0 (Representa {1, 3, 5, 6})

Intersección (A ∩ B): AND bit a bit -> 0 0 0 1 0 0 0 0 (Representa {3})

Diferencia (A - B): A AND (NOT B) -> 0 1 0 0 0 1 0 0 (Representa {1, 5})

FIN